

Attribute-Based Policy-Hiding Redactable Blockchain With Authorizable Verification for Energy Internet

Jingting Xue¹, Liang Liu¹, Fagen Li², *Member, IEEE*, Ximin Jing, Wenzheng Zhang¹,
Xiaojun Zhang¹, and Yu Zhou

Abstract—The goal of the energy internet is to enable communication between distributed energy endpoints to facilitate information exchange and trading matching. Blockchain, which ensures content consistency, bridges the gaps between multiple energy stakeholders and satisfies the need for universal trust. However, applying blockchain directly in peer-to-peer trading can result in the permanent inclusion of illegal or outdated data in blockchain databases, hindering efficient interaction. Driven by emerging application requirements and legal regulations, redactable blockchain (EuroS&P 2017) was introduced, allowing the block history to be rewritten via collisions in chameleon hash functions (CHF). Nevertheless, current solutions still face challenges, including trapdoor security risks, privacy leakage, and indefinite redactions. This article presents a redactable energy blockchain scheme (REBS) with partially attribute-based policy hiding, which enables controlled multiauthority key generation and redactor attribute verification. To prevent the abuse of redaction permission, REBS employs time-updatable CHFs for redaction. Additionally, we design a delegation algorithm that adds dynamic flexibility to attribute verification nodes, permitting them to delegate key share distribution authority to other nodes on demand. The detailed security proofs and performance evaluations demonstrate the feasibility of REBS.

Index Terms—Attribute-based policy hiding, authorizable verification, distributed energy trading, privacy protection, redactable blockchain.

Received 1 October 2024; revised 24 December 2024; accepted 3 May 2025. Date of publication 13 May 2025; date of current version 25 July 2025. This work was supported in part by the Natural Science Foundation of Sichuan Province Under Grant 2023NSFSC1398, Grant 2022YFG0172, Grant 2022JDR0061 and Grant 2025ZNSFSC0495; in part by the Natural Science Starting Project of SWPU Under Grant 2021QHZ017 and Grant 2023QHZ002; in part by the Foundation of Science and Technology of the Communication Security Laboratory of China Under Grant 61421030107012102; and in part by the National Natural Science Foundation of China Under Grant 61902327. (*Jingting Xue and Liang Liu are co-first authors.*) (*Corresponding authors: Jingting Xue; Fagen Li.*)

Jingting Xue, Liang Liu, Ximin Jing, and Xiaojun Zhang are with the School of Computer Science and Software Engineering, Engineering Research Center for Intelligent Oil and Gas Exploration, Southwest Petroleum University, Chengdu 610500, China (e-mail: jtxue@swpu.edu.cn; liuliang202401@163.com; cjhsjxm12833@163.com; zhangxjdzkd2012@163.com).

Fagen Li is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: fagenli@uestc.edu.cn).

Wenzheng Zhang and Yu Zhou are with the Science and Technology on Communication Security Laboratory, Institute of Southwestern Communication, Chengdu 610041, China (e-mail: zzw85169038@sina.com; zhouyu.zhy@tom.com).

Digital Object Identifier 10.1109/IJOT.2025.3569699

I. INTRODUCTION

WITH the rapid growth of the energy internet, peer-to-peer (P2P) energy trading has become a key model for ensuring the efficient bidirectional flow of distributed energy and data [1]. Consequently, blockchain applications in the energy sector have garnered significant attention from academia [2], [3], [4] and industry. For example, the TransActive microgrid project in Brooklyn, New York; the Share&Charge electric vehicle charging station project in Germany; and China's first blockchain-based clean energy project—Shekou Energy Blockchain. As it matures, blockchain is better able to provide secure data sharing for energy trading, whereas consensus mechanisms guarantee robust on-chain security and support transparent regulation. However, the immutability of blockchain also raises concerns about data validity and limits its flexibility in applications.

From a data disclosure perspective, blockchain immutability can mean that erroneous energy trading data or flawed contracts are not corrected [5] and that outdated information remains undeleted. For example, in the highly publicized DAO incident [6], a recursive call vulnerability in a smart contract led to theft of 3.6 million Ether. Similarly, in the Ethereum King incident [7], an attack rendered a contract nonfunctional. As of June 2024, the Bitcoin ledger had grown to 580.70 GB, imposing high storage overheads and performance limitations. Additionally, the read-write and interaction demands of emerging blockchain applications are constrained by immutability. In distributed energy trading, stakeholders need to continuously update shared energy data, authentication information, and smart contract conditions in real time to provide viable energy trading services. In response to these current situations and needs, governments have enacted data security laws promoting flexible redaction and data management. The European Union's general data protection regulation (GDPR) [8] grants users the "right to be forgotten," allowing them to delete or modify personal data at any time. China's personal information protection law (PIPL) [9], enacted in 2021, imposes similar requirements on data storage. Researching blockchain redaction is thus crucial for improving application compatibility and addressing new demands.

The concept of a redactable blockchain [10] emerged to allow the modification of on-chain data through trapdoor management. Current representative redactable blockchain solutions fall into two categories: noncryptography-based and cryptography-based approaches. The former approaches [11],

[12], [13], [14], [15], [16] typically update transactions or blocks through voting, but they suffer from high redaction overheads and struggle with high-frequency redaction requests, and may allow the reconstruction of historical transactions. Cryptography-based solutions effectively mitigate these issues. Specifically, chameleon hash functions (CHFs) replace the traditional hash functions used for block consensus, enabling the trapdoor holder to modify the transaction content without altering the block's hash value, preserving the blockchain structure between blocks. To allow flexible control over redaction permission, Derler et al. [17] introduced the policy-based chameleon hash (PCH) primitive, which supports fine-grained redaction. Building on the PCH, subsequent research [18], [19], [20], [21], [22], [23], [24], [25], [26] improved trapdoor management, redaction authorization, and decentralized redaction, but challenges related to protecting redactor privacy and preventing the abuse of redaction permission remain unresolved. Thus, designing a transaction-level redundant blockchain solution that protects privacy and prevents redaction abuse without relying on a trusted central authority is the next challenge to address.

This article presents a redactable energy blockchain scheme (REBS) featuring time-limited redaction and authorizable verification; this scheme safeguards redactor privacy through partially attribute-based policy hiding. The specific contributions of this study are as follows.

- 1) A REBS is proposed for flexible management of distributed energy trading. It establishes attribute-based redaction policies, creates a mapping between node attributes and trapdoors, and employs CHFs with ephemeral trapdoors for transaction-level redaction. Distributed consensus among stakeholders ensures the consistency of redacted transactions.
- 2) Partially attribute-based policy hiding is implemented. REBS uses a linear secret sharing scheme to enforce attribute-based redaction permissions. A semiopen consortium blockchain supports node privacy by concealing attributes in the policy.
- 3) REBS is expanded to include time-limited redaction and authorizable verification. A time parameter enables time-sensitive transaction redaction and, by linking to predefined redaction permissions, prevents abuse resulting from attribute changes, even if the node previously held the attribute key. Drawing on re-encryption concepts, REBS implements authorizable attribute verification, allowing flexible joining/exiting of verification nodes. Multiauthority attribute-based encryption (MA-ABE) is used for decentralized redaction key distribution.
- 4) A rigorous security analysis is conducted, which demonstrates the robustness of REBS in terms of indistinguishability, public and private collision resistance, and partial policy hiding. Performance evaluations, backed by theoretical analysis and simulations, attest to the feasibility of REBS.

The remainder of this article is organized as follows. Section II reviews related work. Section III introduces a preliminary overview of REBS. Section IV presents a system

overview, including the system model, design goals, security models, and definition and explanation. Section V presents a concrete instantiation of REBS. Sections VI and VII discuss the security and performance of the proposed approach, respectively. Finally, Section VIII concludes this article and future work.

II. RELATED WORK

In response to the need for “removing inappropriate content and ensuring the right to be forgotten,” Ateniese et al. [10] first introduced the concept of a redactable blockchain. Research on redactable blockchains has since expanded, covering both noncryptography-based and cryptography-based solutions.

Noncryptography-Based Solutions: Puddu et al. [11] developed the μ -chain, a blockchain structure supporting the redaction of on-chain transactions by releasing new transaction version numbers, although it does not ensure content consistency. To address this, Deuber et al. [12] proposed a permissionless redaction method in which voting is used to reach consensus on modified transactions. Ren et al. [13] introduced a privacy-preserving solution for the Internet of Things. Notably, these early approaches are limited by high overhead and frequent interactions. Florian et al. [14] proposed a data erasing strategy that supports redaction on local blockchains, whereas Thyagarajan et al. [15] introduced the Reparo protocol, which is based on sidechain technology, as a publicly verifiable redaction layer. However, Reparo only modifies local blockchain content and does not affect the immutability of original transactions on other nodes. Li et al. [16] proposed a universal redactable blockchain protocol without permission settings, compatible with both proof-of-stake and proof-of-work mechanisms. Overall, existing noncryptographic solutions face issues, such as high redaction overheads, inability to handle frequent redaction requests, lack of transaction history security, and vulnerability to historical version reconstruction, rendering them impractical.

Ateniese et al. [10] introduced a coarse-grained redactable blockchain solution using a chameleon hash, which modifies data via hash collisions. Ashritha et al. [34] enhanced security by incorporating nonlinear secret sharing, whereas Li et al. [35] proposed a new CHF granting redaction rights to nodes in consortium blockchain networks. For fine-grained redaction, Derler et al. [17] combined CHFs with ephemeral trapdoors [36] and attribute-based encryption [37] to create a PCH function. This links transactions to access policies, granting redaction rights when nodes meet specific attribute requirements. Subsequent research [18], [19], [20], [21], [22], [23], [24] expanded functionality to include accountability [18], [19], revocability [20], [21], self-management [22], [23], and k -time modification operations [24]. Clearly, trapdoor management is central to PCH-based schemes. Unfortunately, the aforementioned schemes focus primarily on implementation, overlooking inefficiencies and centralization issues related to trapdoor management. Jia et al. [28] introduced a decentralized CHF and designed a distributed redactable blockchain structure. Xu et al. [33] applied redactable blockchains in healthcare, supporting user privacy protection. Li et al. [30]

TABLE I
FUNCTION COMPARISON OF SEVERAL REPRESENTATIVE REDACTABLE BLOCKCHAIN WORKS

Schemes	Technology	Dynamicity	Transaction-level redaction	Scalability	Decentralized redaction permissions	User attributes hidden
Ateniese(2017) [10]	CH	×	×	✓	×	•
Deuber(2019) [12]	Voting	×	×	✓	✓	•
Derler(2019) [17]	CH, CP-ABE	×	✓	✓	×	×
Jia(2021) [22]	CH	×	✓	✓	×	×
Huang(2021) [27]	CH, Ring Signatures	×	×	✓	✓	•
Xu(2021) [24]	CH, Signatures	✓	✓	✓	×	•
Jia(2022) [28]	CH, Accumulators	✓	×	✓	✓	•
Ma(2022) [25]	CHET, CP-ABE	×	✓	✓	✓	×
Zhang(2023) [26]	CHET, CP-ABE	×	✓	✓	✓	×
Shen(2023) [29]	CHET, Accumulators	×	×	×	✓	•
Li(2023) [16]	Voting	×	×	×	✓	•
Shao(2023) [23]	CH	×	✓	✓	✓	•
Xu(2023) [19]	CH, CP-ABE	×	✓	✓	✓	×
Li(2023) [30]	CH, MPC	✓	✓	✓	✓	•
Dai(2024) [31]	CH, Voting	×	×	×	✓	•
Zhang(2024) [32]	CH, DPSS	×	✓	✓	✓	•
Xu(2024) [33]	CH, ABE	×	✓	×	✓	×
Ours	CHET, CP-ABE	✓	✓	✓	✓	✓

i. Dynamicity: the ability for nodes responsible for verifying node attributes to join or leave at any time, supporting validation of authorizations between nodes. ii. Scalability: the compatibility of the proposed redactable blockchain structure with the classical blockchain systems.

introduced a noninteractive CHF that dynamically distributes trapdoor keys within a group, implementing the permissionless Wolverine blockchain system. To improve redaction granularity, Ma et al. [25] proposed a decentralized chameleon hash (DPCH) function based on attribute policies but did not address issues with attribute verification institutions. Zhang et al. [26] combined re-encryption, ciphertext-policy attribute-based encryption, and CHFs to develop a decentralized attribute-based chameleon hash (DACH) function, which supports dynamic node joining. However, none of these schemes consider the risk of user attribute privacy leaks. Additionally, assigning redactors to modify blockchain content introduces security concerns, such as authority abuse and inappropriate or unnecessary modifications.

In summary, current cryptography-based redactable blockchain solutions focus on trapdoor management, redaction authorization, and decentralization, but they fail to address user attribute privacy protection or the abuse of redaction rights. Moreover, most solutions struggle to balance performance and security, making them unsuitable for regulatory P2P energy trading consortium blockchain systems. On the basis of these findings, we propose REBS. Table I compares several representative redactable blockchain solutions with our proposed scheme.

III. PRELIMINARIES

A. Access Structures and Linear Secret Sharing Scheme

Definition 1 (Access Structure): Let $\mathbb{U}$ be a set of attributes. A nonempty collection $\mathbb{A} \subseteq 2^{\mathbb{U}} \setminus \{\emptyset\}$ is called an access structure on $\mathbb{U}$. For any attribute sets B and C , if $B \in \mathbb{A}$ and $B \subseteq C$ implies that $C \in \mathbb{A}$, then $\mathbb{A}$ is called a monotone access structure.

Definition 2 (Linear Secret Sharing Scheme [38]): Given a set of participants $\mathcal{P} = \{P_1, \dots, P_n\}$, a secret sharing scheme Π is called a linear secret sharing scheme if it satisfies the following.

- 1) The participants' secret shares form a vector over $\mathbb{Z}_p$.
- 2) Given a secret $s \in \mathbb{Z}_p$, random numbers $r_2, \dots, r_m \in \mathbb{Z}_p$ are chosen to construct the column vector $\vec{v} = (s, r_2, \dots, r_m)^\top$. On the basis of Π , $A_{\ell \times m} \cdot \vec{v}$ yields ℓ share vectors of s , with share $(A_{\ell \times m} \cdot \vec{v})_i$ distributed to participant $P(i)$. Here, $A_{\ell \times m}$ is the share generation matrix of Π , with elements in $\mathbb{Z}_p$.

Liu et al. proposed [39] to convert any threshold access structure into a corresponding LSSS matrix.¹ For an (t, n) threshold access structure $((\text{attr}_1, t_1), (\text{attr}_2, t_2), \dots, (\text{attr}_n, t_n), t)$, the auxiliary matrix

$$M = \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & 2 & 2^2 & \dots & 2^{t-1} \\ 1 & 3 & 3^2 & \dots & 3^{t-1} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & n & n^2 & \dots & n^{t-1} \end{pmatrix} \quad (1)$$

is used to construct the corresponding LSSS matrix $A_{\ell \times m}$, where each attribute (attr_i, t_i) includes an attribute name attr_i and an attribute value t_i . The mapping function $\rho(i) = \text{attr}_i$ can identify a participant's attribute name on the basis of matrix indices without revealing the attribute value. According to [38], every linear secret sharing scheme exhibits linear reconstruction properties. Let Π be the LSSS corresponding to a monotone access structure $\mathbb{A}$, and let $S \in \mathbb{A}$ be any authorized set.² Define $I = \{i | \rho(i) \in S\}$, with $I \subseteq \{1, \dots, \ell\}$. If λ_i is a valid share of secret s for Π , then there exists a set of constants $\{\mu_i \in \mathbb{Z}_p\}_{i \in I}$ ³ such that $\sum_{i \in I} \mu_i A_i = (1, 0, \dots, 0)$, where μ_i satisfies $\sum_{i \in I} \mu_i \lambda_i = s$, and A_i is the i th row of the matrix $A_{\ell \times m}$.

¹Compared with the Lewko-Waters algorithm [39], [40] avoids converting a (t, n) threshold string into an AND-OR access tree, reducing preprocessing overheads and producing a smaller, more efficient LSSS matrix.

²A set in $\mathbb{A}$ is called authorized, whereas a set not in $\mathbb{A}$ is called unauthorized.

³On the basis of matrix A , the set of constants $\{\mu_i\}_{i \in I}$ can be computed in polynomial time [38]. For unauthorized sets, no such constants exist.

B. Predicate Encryption

Katz et al. [41] first introduced predicate encryption (PE), which links keys with predicates and ciphertexts with attributes. A user can decrypt the ciphertext only if the result of evaluating their private key predicate on the ciphertext's attributes is 1. PE is defined as a four-tuple (Setup, Encrypt, KeyGen, Decrypt). PE guarantees attribute hiding [41] against any probabilistic polynomial time (PPT) adversary $\mathcal{P}$, with the adversary's advantage being negligible, i.e., $\text{Adv}_{\mathcal{P}, \text{PE}}^{\text{AttrHid}}(\lambda) \leq f(\lambda)$.

C. Multiauthority Attribute-Based Encryption

Lewko and Waters [40] introduced MA-ABE, which supports fine-grained access control and decentralized key distribution. MA-ABE is defined by a five-tuple (GPSetup, AASetup, Encrypt, KeyGen, Decrypt). MA-ABE provides static security (indistinguishability under chosen-ciphertext attack, IND-CCA2)⁴ [40] against any PPT adversary $\mathcal{M}$. The adversary's advantage in the IND-CCA2 experiment is negligible, i.e., $\text{Adv}_{\mathcal{M}, \text{MA-ABE}}^{\text{IND-CCA2}}(\lambda) \leq f(\lambda)$.

D. Chameleon Hash With Ephemeral Trapdoor

Camenisch et al. [36] introduced CHF with an ephemeral trapdoor (CHET), which is designed to resist key exposure attacks. CHET is defined as a five-tuple (GPGen, KeyGen, CHash, ChVer, ChCld). Under the RSA-like assumption, CHET offers strong indistinguishability, public collision resistance, and private collision resistance against any PPT adversary $\mathcal{C}$ [36]. The adversary's advantage in the corresponding experiments is negligible, i.e., $\text{Adv}_{\mathcal{C}, \text{CHET}}^{\text{SIND}}(\lambda) \leq f(\lambda)$, $\text{Adv}_{\mathcal{C}, \text{CHET}}^{\text{PubCR}}(\lambda) \leq f(\lambda)$, and $\text{Adv}_{\mathcal{C}, \text{CHET}}^{\text{PriCR}}(\lambda) \leq f(\lambda)$.

IV. SYSTEM OVERVIEW

A. System Model

As shown in Fig. 1, the system model involves the attribute management center ($\mathcal{AMC}$), attribute verification nodes ($\mathcal{AVNs}$), transaction senders ($\mathcal{TS}$ s), and transaction redactors ($\mathcal{TR}$ s). The model also includes a redactable consortium blockchain ($\mathcal{RCB}$) for energy trading, which supports on-demand updates of on-chain information and publishes the latest consensus on energy data to stakeholders.

- 1) $\mathcal{TS}$ s are the creators and initiators of transactions on the consortium blockchain. They define transaction redaction policies, including attribute requirements and time constraints.
- 2) $\mathcal{TR}$ s are entities that meet the attribute requirements specified in the redaction policies; typically, $\mathcal{TR}$ s are stakeholders needing to redact transactions.
- 3) $\mathcal{AVNs}$ act as consensus nodes within the blockchain network. They distribute attribute key shares to $\mathcal{TR}$ s on the basis of attributes and validate redacted transactions to reach block consensus. $\mathcal{AVNs}$ are considered semi-honest and may attempt to recover attribute keys (e.g.,

⁴Compared with IND-CCA1 (indistinguishability under nonadaptive selective ciphertext attack), IND-CCA2 offers stronger security.

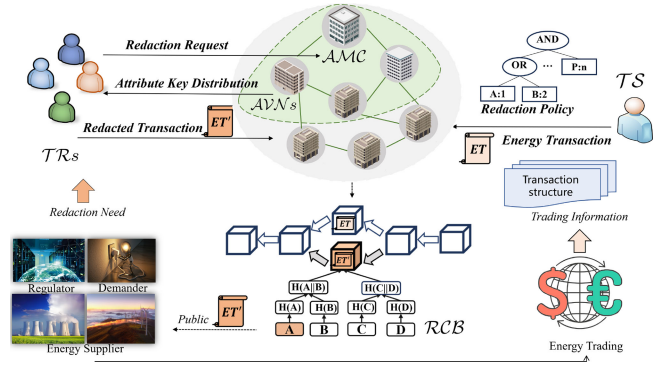


Fig. 1. System model.

through collusion attacks) to gain redaction rights for personal benefit. $\mathcal{AVNs}$ can dynamically join/leave the network.

- 4) The $\mathcal{AMC}$ is a trusted entity responsible for distributing chameleon hash keys to $\mathcal{TR}$ s that meet redaction policy requirements and oversee the legality of $\mathcal{AVNs}$ ' actions during protocol execution. For scalability, the $\mathcal{AMC}$ can be represented by highly reputable $\mathcal{AVNs}$.

B. Design Goals

Fine-Grained Redaction: Within the redaction period specified by any $\mathcal{TS}$, $\mathcal{TR}$ s should be able to perform transaction-level redactions on the energy blockchain.

Dynamic Participation: $\mathcal{AVNs}$ should be able to dynamically join and leave the network, ensuring forward and backward security without affecting attribute verification or transaction redaction accuracy.

Consensus Correctness: Network nodes should reach consensus on the hash values generated by $\mathcal{TS}$ s and the chameleon hash values generated by $\mathcal{TR}$ s.

Indistinguishability: Adversaries should not be able to determine whether a random number in a block originates from a hash generation algorithm or a hash collision algorithm. Postredaction hash values should be indistinguishable from the original transaction hashes.

Collision resistance: Only $\mathcal{TR}$ s meeting the redaction policies should be able to find hash collisions; other nodes should not.

Privacy Protection: $\mathcal{TR}$ s complying with redaction policies should not reveal attribute values containing sensitive user information.

C. Security Models

Referring to [17], we define the following security model for indistinguishability, public collision resistance, private collision resistance, and partial policy hiding under the design goals outlined above.

1) **Security Model for Indistinguishability:** Any PPT adversary $\mathcal{A}$ should find it difficult to distinguish whether the random number r is generated by **CHash** or **ChCld**. In other words, $\mathcal{A}$ should not be able to tell whether a transaction hash is redacted or original. Formally, if the probability that $\mathcal{A}$ wins *Experiment 1* is negligible, i.e., $\text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{SIND}}(\lambda) =$

$Pr[\text{IND}_{\mathcal{A}}^{\text{REBS}}(\lambda) = 1] - [1/2] \leq f(\lambda)$, then it is said to have indistinguishability.

Experiment 1: $\text{IND}_{\mathcal{A}}^{\text{REBS}}(\lambda)$
 run **GPGen**, **KeyGen**, **LSSSConvert**;
 $\mathcal{OR} = \{\text{CHash}, \text{ChCld}\};$
 $(h, t, r, \text{Info}_{\text{Trap}}) \leftarrow \text{CHash}(pk_{\mathcal{AMC}}, Tx, (A_{\ell \times m}, \rho, \mathcal{T}));$
 $(h, t, \tilde{r}, \text{Info}_{\text{Trap}}) \leftarrow \text{ChCld}(k_{\mathcal{JR}}, K_{\text{Attr}-\mathcal{JR}}, Tx, h, t, r, \text{Info}_{\text{Trap}}, \tilde{Tx}).$
 $\tilde{b} \leftarrow \mathcal{A}^{\mathcal{OR}}(pk_{\mathcal{AMC}}, sk_{\mathcal{AVN}}, k_{\mathcal{JR}}, b)$, where $b \in \{0, 1\}$.

In *Experiment 1*, if $\tilde{r} = \perp \vee \tilde{r} = \perp$, return $\perp$; if $b = 0$, return $(h, t, \tilde{r}, \text{Info}_{\text{Trap}})$; if $b = 1$, return $(h, t, \tilde{r}, \text{Info}_{\text{Trap}})$. If $\tilde{b} = b$, return 1; otherwise, return 0.

2) *Security Model for Public Collision Resistance*: A PPT adversary $\mathcal{A}$ should not be able to find new random numbers satisfying **ChCld** by querying the oracle $\mathcal{OR}(\text{ChCld})$. Formally, the probability that $\mathcal{A}$ finds a hash collision in *Experiment 2* is negligible, i.e., $\text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{PubCR}}(\lambda) = Pr[\text{PubCollRes}_{\mathcal{A}}^{\text{REBS}}(\lambda) = 1] \leq f(\lambda)$, then it is said to have public collision resistance.

Experiment 2: $\text{PubCollRes}_{\mathcal{A}}^{\text{REBS}}(\lambda)$
 run **GPGen**, **KeyGen**, **LSSSConvert**;
 $\mathcal{OR} = \{\text{AttrKeyGen}, \text{ChCld}\};$
 $k_{\text{Attr}-\mathcal{JR}} \leftarrow \text{AttrKeyGen}(ID_{\mathcal{JR}}, (\text{attr}_{\mathcal{JR}}, t_{\mathcal{JR}}), sk_{\mathcal{AVN}}, \text{Sig}_{\mathcal{AMC}});$
 If $\text{ChVer}(Tx, h, t, r) \neq 1 \vee (j, K_{\text{Attr}-\mathcal{JR}}) \notin \mathcal{Q}$, return $\perp$;
 Otherwise, return $\tilde{r} \leftarrow \text{ChCld}(k_{\mathcal{JR}}, K_{\text{Attr}-\mathcal{JR}}, Tx, h, t, r, \text{Info}_{\text{Trap}}, \tilde{Tx}).$
 $(Tx_0, h_0, t_0, \text{Info}_{\text{Trap}_0}, \tilde{Tx}_0, \tilde{r}_0) \leftarrow \mathcal{A}^{\mathcal{OR}}(pk_{\mathcal{AMC}}).$

In *Experiment 2*, if $\text{ChVer}(pk_{\mathcal{AMC}}, Tx_0, h_0, t_0, r_0) = 1 \wedge \text{ChVer}(pk_{\mathcal{AMC}}, \tilde{Tx}_0, h_0, t_0, \tilde{r}_0) = 1 \wedge (Tx_0 \notin \mathcal{TX}) \wedge (Tx_0 \neq \tilde{Tx}_0)$, return 1; otherwise, return 0.

3) *Security Model for Private Collision Resistance*: When oracle $\mathcal{OR}(\text{AttrKeyGen}, \text{ChCld})$ is adaptively accessible, any PPT adversary $\mathcal{A}$ should find it difficult to find a new random number that satisfies **ChCld** under the redaction policy. Formally, if the probability of $\mathcal{A}$ finding a hash collision in *Experiment 3* is negligible, i.e., $\text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{PriCR}}(\lambda) = Pr[\text{PriCollRes}_{\mathcal{A}}^{\text{REBS}}(\lambda) = 1] \leq f(\lambda)$, then it is said to have private collision resistance.

Experiment 3: $\text{PriCollRes}_{\mathcal{A}}^{\text{REBS}}(\lambda)$
 run **GPGen**, **KeyGen**, **LSSSConvert**;
 $\mathcal{OR} = \{\text{AttrKeyGen}, \text{AttrKeyGen}, \text{CHash}, \text{ChCld}\};$
 $k_{\text{Attr}-\mathcal{JR}} \leftarrow \text{AttrKeyGen}(ID_{\mathcal{JR}}, (\text{attr}_{\mathcal{JR}}, t_{\mathcal{JR}}), sk_{\mathcal{AVN}}, \text{Sig}_{\mathcal{AMC}});$
 $k_{\text{Attr}-\mathcal{JR}} \leftarrow \text{AttrKeyGen}(ID_{\mathcal{JR}}, (\text{attr}_{\mathcal{JR}}, t_{\mathcal{JR}}), sk_{\mathcal{AVN}}, \text{Sig}_{\mathcal{AMC}});$
 $(h, t, r, \text{Info}_{\text{Trap}}) \leftarrow \text{CHash}(pk_{\mathcal{AMC}}, Tx, (A_{\ell \times m}, \rho, \mathcal{T}));$
 If $(j, K_{\text{Attr}-\mathcal{JR}}) \notin \mathcal{Q}$, return $\perp$;
 Otherwise, return $\tilde{r} \leftarrow \text{ChCld}(k_{\mathcal{JR}}, K_{\text{Attr}-\mathcal{JR}}, Tx, h, t, r, \text{Info}_{\text{Trap}}, \tilde{Tx}).$
 $(Tx_0, h_0, t_0, \text{Info}_{\text{Trap}_0}, \tilde{Tx}_0, \tilde{r}_0) \leftarrow \mathcal{A}^{\mathcal{OR}}(pk_{\mathcal{AMC}}).$

In *Experiment 3*, if $\text{ChVer}(pk_{\mathcal{AMC}}, Tx_0, h_0, t_0, r_0) = 1 \wedge \text{ChVer}(pk_{\mathcal{AMC}}, \tilde{Tx}_0, h_0, t_0, \tilde{r}_0) = 1 \wedge (h, \cdot, A_{\ell \times m}, \rho, \mathcal{T}) \in \mathcal{H} \wedge (Tx_0 \neq \tilde{Tx}_0) \wedge (\{\text{Attr}, \mathcal{T}\} \cap \mathcal{V} = \emptyset) \wedge (h, \tilde{Tx}, \cdot, \cdot, \cdot) \notin \mathcal{H}$, return 1; otherwise, return 0.

4) *Security Model for Partially Policy Hiding*: If any PPT adversary $\mathcal{A}$ has a negligible probability of winning *Game* against challenger $\mathcal{B}$, i.e., if $\text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{ParPH}}(\lambda) = |Pr[\text{PartPolHid}_{\mathcal{A}}^{\text{REBS}}(\lambda) = 1] - (1/2)| \leq f(\lambda)$, then it is considered to have partial policy hiding.

Game $\text{PartPolHid}_{\mathcal{A}}^{\text{REBS}}(\lambda)$:

- 1) *Setup*⁵: $\mathcal{B}$ runs **GPGen** to obtain the global parameters gp and $(pk_{\mathcal{AMC}}, sk_{\mathcal{AMC}})$. $\mathcal{B}$ then executes **Key $\mathcal{JR}$ Gen** to generate $k_{\mathcal{JR}}$ and **Key $\mathcal{AVN}$ Gen** to generate $(pk_{\mathcal{AVN}}, sk_{\mathcal{AVN}})$.
- 2) *Query Phase 1*⁶: $\mathcal{A}$ adaptively queries $\mathcal{B}$ for attribute keys. For each query, $\mathcal{A}$ sends an attribute set $(\text{Attr}_{\mathcal{JR}}, \mathcal{T}_{\mathcal{JR}})$ and an identifier $ID_{\mathcal{JR}}$ to $\mathcal{B}$, which then generates the attribute key $k_{\text{Attr}-\mathcal{JR}}$ and the attribute key set $K_{\text{Attr}-\mathcal{JR}} = \{k_{\text{Attr}-\mathcal{JR}}\}_{i \in \text{Attr}_{\mathcal{JR}}}$ and returns it to $\mathcal{A}$, ensuring that each $ID_{\mathcal{JR}}$ is distinct.
- 3) *Challenge*: $\mathcal{A}$ runs **LSSSConvert** to generate two redaction policies, $\mathbb{A}_0 = (A_{\ell \times m}, \rho, \mathcal{T}_0)$ and $\mathbb{A}_1 = (A_{\ell \times m}, \rho, \mathcal{T}_1)$. $\mathcal{A}$ sends $\mathbb{A}_0$, $\mathbb{A}_1$, and two transactions Tx_0 and Tx_1 of the same length to $\mathcal{B}$. In response, $\mathcal{B}$ randomly selects a bit $\beta \in \{0, 1\}$ and runs **CHash** $(pk_{\mathcal{AMC}}, Tx_\beta, (A_{\ell \times m}, \rho, \mathcal{T}_\beta))$, obtaining $(h_\beta, t_\beta, r_\beta, \text{Info}_{\text{Trap}_\beta})$, which is sent to $\mathcal{A}$ as the challenge ciphertext.
- 4) *Query Phase 2*: $\mathcal{A}$ continues querying $\mathcal{B}$ for keys corresponding to attribute sets and global identifiers.
- 5) *Guess*: $\mathcal{A}$ outputs a guess bit $\tilde{\beta} \in \{0, 1\}$. If $\tilde{\beta} = \beta$, $\mathcal{A}$ wins and returns 1; otherwise, it returns 0.

In the game, the probability that $\mathcal{A}$ wins is represented as $\text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{ParPH}}(\lambda) = |Pr[\tilde{\beta} = \beta] - (1/2)| \leq f(\lambda)$.

D. Definition and Explanation

Definition 3: The RSA key generation algorithm is represented as $(n, p, q, \check{e}, \check{d}) \leftarrow \text{RSAKeyGen}(1^\lambda)$. Given the security parameter λ , **RSAKeyGen** outputs large primes p and q , a constant $\check{e}$, and its inverse $\check{d}$ such that $\check{e}\check{d} \equiv 1 \pmod{\varphi(n)}$, where $n = pq$ and $\varphi(\cdot)$ is Euler's totient function.

Definition 4: The bilinear pair generation algorithm is represented as $(N, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \text{BilGen}(1^\lambda)$. Given λ , **BilGen** outputs two multiplicative cyclic groups $\mathbb{G}$ and $\mathbb{G}_T$ of order N and a bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$. Here, $N = p_1 p_2 p_3$, $\mathbb{G}_{p_1} = \langle g \rangle$, and $\mathbb{G} = \mathbb{G}_{p_1} \times \mathbb{G}_{p_2} \times \mathbb{G}_{p_3}$, with p_1 , p_2 , and p_3 as the orders of the subgroups $\mathbb{G}_{p_1}$, $\mathbb{G}_{p_2}$, and $\mathbb{G}_{p_3}$, respectively.

Definition 5: The policy transformation algorithm is represented as $(A_{\ell' \times m'}, L') \leftarrow \text{Convert}(A_{\ell \times m}, \bar{L})$, where $\bar{L} = (L_1, L_2, \dots, L_\ell)$ contains attributes or threshold strings. Given

⁵Since the number of $\mathcal{AVN}$ and $\mathcal{JR}$ in this scheme does not affect security, $\mathcal{B}$ can replace all $\mathcal{AVN}$ in the security proof.

⁶The attribute sets from *Query Phase 1* must not satisfy redaction policies $\mathbb{A}_0$ or $\mathbb{A}_1$.

matrix $A_{\ell \times m}$ and vector $\vec{L}$, **Convert** outputs the transformed matrix $A_{\ell' \times m'}$ and vector $\vec{L}'$, with the following process.

- 1) Iterate through $\vec{L}$ to find the first index z where the element is a threshold string rather than an attribute, i.e., a threshold string $L_z : F_z = (F_{z,1}, F_{z,2}, \dots, F_{z,\ell_2}, m_2)$ exists. If none is found, return $(A_{\ell \times m}, L)$.
- 2) Analyze F_z to obtain its ℓ_2 subthreshold strings $F_{z,1}, F_{z,2}, \dots, F_{z,\ell_2}$ and threshold m_2 .
- 3) For the (m_2, ℓ_2) threshold access structure, construct the corresponding LSSS matrix per formula (1), then insert this matrix into row z of $A_{\ell \times m}$ to obtain the new matrix $A_{\ell' \times m'}$ and vector $\vec{L}' = (L_1, L_2, \dots, F_{z,1}, \dots, F_{z,\ell_2}, \dots, L_\ell)$, where $\ell' = \ell + \ell_2 - 1$ and $m' = m + m_2 - 1$. Finally, return $(A_{\ell' \times m'}, \vec{L}')$.

Here, we formally describe the REBS redaction policy as an LSSS policy matrix, including symbols and their relationships. Let $(A_{\ell \times m}, \rho, \mathcal{T})$ represent the redaction policy, involving m attribute categories (each node can have up to m attributes). Each attribute includes the attribute name $\rho(i)$ and value $t_{\rho(i)}$. $A_{\ell \times m}$ is the policy matrix, ρ is the mapping function that associates each row A_i of $A_{\ell \times m}$ with the attribute name $\rho(i)$, and $\mathcal{T}$ is the set of attribute values $(t_{\rho(1)}, \dots, t_{\rho(\ell)})$, where $t_{\rho(i)} \in \mathbb{Z}_N$. In REBS, the transaction redactor attribute set $(ID_{\mathcal{T}\mathcal{R}})$ is $(Attr_{\mathcal{T}\mathcal{R}}, \mathcal{T}_{\mathcal{T}\mathcal{R}})$, where $Attr_{\mathcal{T}\mathcal{R}} \subseteq \mathbb{Z}_N$ represents the attribute names and where $\mathcal{T}_{\mathcal{T}\mathcal{R}} = \{t_{\mathcal{T}\mathcal{R}_i}\}_{i \in Attr_{\mathcal{T}\mathcal{R}}}$ represents the attribute values. To protect privacy, REBS hides the set of attribute values $\mathcal{T}$, while $(A_{\ell \times m}, \rho)$ is sent openly with the ciphertext.

V. PROPOSED REBS

A. System Initialization Phase

GPGen: $\mathcal{AMC}$ generates public/private keys and global parameters.

- 1) Run $(n, p, q) \leftarrow \text{RSAKeyGen}(1^\lambda)$, where λ is the security parameter, p and q are large primes, and $n = pq$. Select a random number $\alpha \in \mathbb{Z}_N$ to generate the private key $sk_{\mathcal{AMC}} = (p, q, \alpha)$.
- 2) Run $(N, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \text{BilGen}(1^\lambda)$ to generate the public key $pk_{\mathcal{AMC}} = (n, g^\alpha)$, where $N = p_1 p_2 p_3$ and where p_1, p_2 , and p_3 are large primes. $\mathbb{G}$ and $\mathbb{G}_T$ are multiplicative cyclic groups, e is the bilinear pairing operation, and g is a generator of $\mathbb{G}$.
- 3) Choose a large prime $\check{e} > \check{n}$, where $\check{n} = \max\{n | n \leftarrow \text{RSAKeyGen}(1^\lambda)\}$. Select secure hash functions $H : \mathbb{G}_T \rightarrow \{0, 1\}^*$, $H_{\mathcal{GD}} : \{0, 1\} \times \mathcal{ID} \rightarrow \mathbb{G}$, and $H_{\text{sig}} : \{0, 1\}^* \times \mathbb{Z}_N \times \mathcal{ID} \rightarrow \mathbb{G}$, where $\mathcal{ID}$ represents user identifiers. The encoding function **encode** : $\{0, 1\}^* \rightarrow \mathbb{G}_T$ and decoding function **decode** : $\mathbb{G}_T \rightarrow \{0, 1\}^*$ are chosen, and the global parameters $gp = \{(N, \mathbb{G}, \mathbb{G}_T, e, g), \check{e}, H, H_{\mathcal{GD}}, H_{\text{sig}}, \text{encode}, \text{decode}\}$ are generated.

Key $\mathcal{T}\mathcal{S}$ Gen: $\mathcal{T}\mathcal{S}$ generates public and private keys.

- 1) Choose a random number $\vartheta \in \mathbb{Z}_N$ to generate the private key $sk_{\mathcal{T}\mathcal{S}} = \vartheta$ and the public key $pk_{\mathcal{T}\mathcal{S}} = g^\vartheta$.

Key $\mathcal{T}\mathcal{R}$ Gen: $\mathcal{AMC}$ generates a chameleon hash key and signature for $\mathcal{T}\mathcal{R}$.

- 1) Upon receiving a registration request from $\mathcal{T}\mathcal{R}$, the chameleon hash key $k_{\mathcal{T}\mathcal{R}} = (p, q)$ and the signature $Sig_{\mathcal{AMC}} = (H_{\mathcal{GD}}(1, ID_{\mathcal{T}\mathcal{R}}))^\alpha$ are generated.
 - 2) Send $(k_{\mathcal{T}\mathcal{R}}, Sig_{\mathcal{AMC}})$ to $\mathcal{T}\mathcal{R}$ via a secure channel.
- Key $\mathcal{AVN}$ Gen**: $\mathcal{AVN}$ generates public and private keys.
- 1) For each $\mathcal{AVN}_i$, random numbers $\varepsilon_i, \eta_i \in \mathbb{Z}_N$ are selected to generate the private key $sk_{\mathcal{AVN}_i} = (\varepsilon_i, \eta_i)$ and the public key $pk_{\mathcal{AVN}_i} = (e(g, g)^{\varepsilon_i}, g^{\eta_i})$.

B. Transaction Generation Phase

LSSSConvert: $\mathcal{T}\mathcal{S}$ creates the redaction policy tree $\mathbb{A}$ and generates the corresponding threshold string $F_{\mathbb{A}}$, converting it into the LSSS matrix $A_{\ell \times m}$.

- 1) Define $\vec{L} = (L_1, L_2, \dots, L_\ell)$, where L_i corresponds to the i th row of $A_{\ell \times m}$, representing an attribute in $\mathbb{A}$.
- 2) Initialize the policy matrix $A_{\ell \times m} = [1]_{1 \times 1}$ and the vector $\vec{L} = [F_{\mathbb{A}}]$.
- 3) Run $(A_{\ell' \times m'}, \vec{L}') \leftarrow \text{Convert}(A_{\ell \times m}, \vec{L})$ to convert $A_{\ell \times m}$ to $A_{\ell' \times m'}$ and $\vec{L}$ to $\vec{L}'$. This process is repeated until all the elements of $\vec{L}$ correspond to attributes in the redaction policy.
- 4) Define the redaction policy $\mathbb{RP} : (A_{\ell \times m}, \rho, \mathcal{T})$, where ρ maps the i th row of $A_{\ell \times m}$ to the i th attribute in $F_{\mathbb{A}}$ and where $\mathcal{T}$ is the set of attribute values.

CHash: $\mathcal{T}\mathcal{S}$ generates and broadcasts the original energy transaction and its chameleon hash tuple.

- 1) Run $(\tilde{n}, \tilde{p}, \tilde{q}) \leftarrow \text{RSAKeyGen}(1^\lambda)$, where $\tilde{n} = \tilde{p}\tilde{q}$ and $\text{gcd}(n, \tilde{n}) = 1$. Execute $\text{Trap} \leftarrow \text{encode}(\tilde{p}, \tilde{q})$ to get an ephemeral trapdoor Trap and its hash $h_{\text{Trap}} = H(\text{Trap})$. Keep Trap confidential and publish $\tilde{n}$.
- 2) select a secure hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_{\tilde{n}}^*$ and a random number $r \in \mathbb{Z}_{\tilde{n}}^*$ and set a global time interval $\Delta t \in \mathbb{Z}_{\tilde{n}}^*$. Generate the original energy transaction Tx , and compute the chameleon hash value $h = \mathcal{H}(Tx)^t r^{\check{e}} \text{ mod } (n\tilde{n})$, where $\check{e} > \check{n}$ and t is the timestamp.
- 3) choose random vectors $\vec{v} = (s, v_2, \dots, v_m)^\top \in \mathbb{Z}_N^m$ and $\vec{\omega} = (0, \omega_2, \dots, \omega_m)^\top \in \mathbb{Z}_N^m$. Compute $\vec{\lambda} = A_{\ell \times m} \cdot \vec{v} = (\lambda_1, \lambda_2, \dots, \lambda_\ell)^\top$ and $\vec{\theta} = A_{\ell \times m} \cdot \vec{\omega} = (\theta_1, \theta_2, \dots, \theta_\ell)^\top$. For each row A_i of $A_{\ell \times m}$, choose a random number $r_i \in \mathbb{Z}_N$, and generate the trapdoor auxiliary information

$$\text{InfoTrap} = ((A_{\ell \times m}, \rho), c_0, \{c_{1,l}, c_{2,l}, c_{3,l}\}_{1 \leq l \leq \ell}, h_{\text{Trap}})$$

where $c_{1,l} = e(g, g)^{\lambda_l} e(g, g)^{\varepsilon_{\rho(l)} t_{\rho(l)} r_l}$, $c_{2,l} = g^{r_l}$, $c_{3,l} = g^{r_l \eta_{\rho(l)}} g^{\theta_l}$, and $c_0 = \text{Trap} \cdot e(g, g)^s$.

- 4) Broadcast the chameleon hash tuple $(h, t, r, \text{InfoTrap})$ and transaction Tx to $\mathcal{AVNs}$, with r as the random number in the block.

ChVer: $\mathcal{AVNs}$ validates the chameleon hash tuple $(h, t, r, \text{InfoTrap})$ and reaches a consensus on Tx .

- 1) Verify h and $r \in \mathbb{Z}_{\tilde{n}}^*$ before $t + \Delta t$. Check $h \stackrel{?}{=} \mathcal{H}(Tx)^t r^{\check{e}} \text{ mod } (n\tilde{n})$. If the equality holds, set the verification result $ver = 1$, indicating consensus on Tx ; otherwise, set $ver = 0$.

C. Transaction Redaction Phase

AttrKeyGen: On the basis of the attribute set ($Attr_{\mathcal{T}\mathcal{R}}, \mathcal{T}\mathcal{R}$), $\mathcal{AVN}_i$ distributes the corresponding attribute key to $\mathcal{T}\mathcal{R}$.

- 1) Verify the signature $Sig_{\mathcal{AMC}}$ by checking $e(g, Sig_{\mathcal{AMC}}) \stackrel{?}{=} e(g^\alpha, H_{g\mathcal{D}}(1, ID_{\mathcal{T}\mathcal{R}}))$. If true, proceed; otherwise, return $\perp$.
- 2) For each attribute $attr_{\mathcal{T}\mathcal{R}_i} \in Attr_{\mathcal{T}\mathcal{R}}$ and value $t_{\mathcal{T}\mathcal{R}_i} \in \mathcal{T}\mathcal{R}$, generate the attribute key $k_{attr_i-\mathcal{T}\mathcal{R}} = g^{\varepsilon_i t_{\mathcal{T}\mathcal{R}_i}} h_{g\mathcal{D}}^{\eta_i}$, and send $k_{attr_i-\mathcal{T}\mathcal{R}}$ to $\mathcal{T}\mathcal{R}$, where $h_{g\mathcal{D}} = H_{g\mathcal{D}}(0, ID_{\mathcal{T}\mathcal{R}})$ and $sk_{\mathcal{AVN}_i} = (\varepsilon_i, \eta_i)$.

ChCld: Based on the attribute key set, the $\mathcal{T}\mathcal{R}$ redacts the target transaction on-chain.

- 1) Check whether the attribute set ($Attr_{\mathcal{T}\mathcal{R}}, \mathcal{T}\mathcal{R}$) satisfies the redaction policy $\mathbb{RP} : (A_{\ell \times m}, \rho, \mathcal{T})$. Let $\mathcal{L}_{A,\rho}$ represent the set of rows in the matrix $A_{\ell \times m}$ corresponding to $Attr_{\mathcal{T}\mathcal{R}}$. Then:
 - a) With trapdoor auxiliary information $Info_{\text{Trap}}$, compute

$$Trap' = \frac{c_0}{\prod_{l \in \mathcal{L}_{A,\rho}} \left(\frac{c_{1,l} \cdot e(H_{g\mathcal{D}}(0, ID_{\mathcal{T}\mathcal{R}}), c_{3,l})}{e(k_{attr_{\rho(l)}-\mathcal{T}\mathcal{R}}, c_{2,l})} \right)^{\mu_l}}$$

where μ_l satisfies $\sum_{l \in \mathcal{L}_{A,\rho}} \mu_l A_l = (1, 0, \dots, 0)$.

- b) Verify $h_{\text{Trap}} \stackrel{?}{=} H(Trap')$. If true, proceed; otherwise, return $\perp$.
- 2) Compute $\varphi(n) = (p-1)(q-1)$ and $\varphi(\tilde{n}) = (\tilde{p}-1)(\tilde{q}-1)$, where the chameleon hash key is $k_{\mathcal{T}\mathcal{R}} = (p, q)$ and $(\tilde{p}, \tilde{q}) \leftarrow \text{decode}(Trap')$. Then, compute $\tilde{d}$ such that $\tilde{e}\tilde{d} \equiv 1 \pmod{\varphi(\tilde{n})}$.
- 3) Compute the new random number $\tilde{r} = (h \cdot (\mathcal{H}(\tilde{T}x)^t)^{-1})^{\tilde{d}} \pmod{\varphi(\tilde{n})}$.
- 4) Run **ChVer**($pk_{\mathcal{AMC}}, \tilde{T}x, h, t, \tilde{r}$). If it outputs 1, indicating that the new transaction is valid, broadcast the updated chameleon hash tuple $(h, t, \tilde{r}, Info_{\text{Trap}})$ and the new transaction $\tilde{T}x$ to the $\mathcal{AVN}$ s; otherwise, return $\perp$.

TimeUpdate: According to the global time interval Δt , $\mathcal{T}\mathcal{S}$ updates the random number r , ensuring that only authorized nodes can modify transactions within a fixed time frame.

- 1) Compute $\varphi(n) = (p-1)(q-1)$ and $\varphi(\tilde{n}) = (\tilde{p}-1)(\tilde{q}-1)$, where $k_{\mathcal{T}\mathcal{R}} = (p, q)$ and $(\tilde{p}, \tilde{q}) \leftarrow \text{decode}(Trap)$. Compute $\tilde{d}$ such that $\tilde{e}\tilde{d} \equiv 1 \pmod{\varphi(\tilde{n})}$.
- 2) Compute the time-updated random number $r' = r \cdot ((\mathcal{H}(Tx)^{\Delta t})^{-1})^{\tilde{d}} \pmod{\varphi(\tilde{n})}$, and generate $Sig_{\mathcal{T}\mathcal{S}} = (H_{\text{sig}}(Tx, r', ID_{\mathcal{T}\mathcal{S}}))^{\vartheta}$.
- 3) Run **ChVer** to verify the validity of the time update. If it outputs 1, send the transaction Tx , the updated chameleon hash tuple $(h, t + \Delta t, r', Info_{\text{Trap}})$, and the signature $Sig_{\mathcal{T}\mathcal{S}}$ to the $\mathcal{AVN}$ s; otherwise, output $\perp$.

D. Authorization Phase

Delegate: Perform the secure removal of attribute validation node $\mathcal{AVN}_i$ and the secure joining of the new node $\widetilde{\mathcal{AVN}_i}$.

- 1) For each $\mathcal{T}\mathcal{R}_j$ that has the attribute $\mathcal{AVN}_i - attr_i$, where $\rho(l) = attr_{\mathcal{AVN}_i} = attr_{\mathcal{T}\mathcal{R}_j}$ in the policy matrix, compute $\sigma_1 = g^{\varepsilon_i t_{\mathcal{T}\mathcal{R}_j}}$, $\sigma_2 = g^{r_i \eta_i}$, and $h'_{g\mathcal{D}} = H_{g\mathcal{D}}(0, ID_{\mathcal{T}\mathcal{R}_j})$,

where $sk_{\mathcal{AVN}_i} = (\varepsilon_i, \eta_i)$ and $t_{\mathcal{T}\mathcal{R}_j}$ correspond to $attr_{\mathcal{T}\mathcal{R}_j}$.

- 2) Verify $e(k_{attr_{\rho(l)}-\mathcal{T}\mathcal{R}_j}, c_{2,l}) \stackrel{?}{=} e(\sigma_1, c_{2,l}) \cdot e(h'_{g\mathcal{D}}, \sigma_2)$. If it is valid, choose random numbers $\tilde{\varepsilon}_i, \tilde{\eta}_i \in \mathbb{Z}_N$ such that $\tilde{\varepsilon}_i \neq \varepsilon_i$ and $\tilde{\eta}_i \neq \eta_i$. Then, generate new trapdoor auxiliary information for $\mathcal{T}\mathcal{R}_j$

$$Info_{\text{Trap}-\mathcal{T}\mathcal{R}_j} = (A_{\ell \times m}, \rho), \{\tilde{c}_0, \tilde{c}_{1,l}, \tilde{c}_{2,l}, \tilde{c}_{3,l}\}, \tilde{h}_{\text{Trap}}$$

where $\tilde{c}_0 = c_0$, $\tilde{c}_{1,l} = c_{1,l} \cdot e(c_{2,l}, g^{\tilde{\varepsilon}_i t_{\mathcal{T}\mathcal{R}_j}}) / e(c_{2,l}, \sigma_1)$, $\tilde{c}_{2,l} = c_{2,l}$, $\tilde{c}_{3,l} = c_{3,l} \cdot (c_{2,l})^{\tilde{\eta}_i} / \sigma_2$, and $\tilde{h}_{\text{Trap}} = h_{\text{Trap}}$.

- 3) Run $k_{attr_i-\mathcal{T}\mathcal{R}_j} \leftarrow \text{AttrKeyGen}(ID_{\mathcal{T}\mathcal{R}_j}, (attr_{\mathcal{T}\mathcal{R}_j}, t_{\mathcal{T}\mathcal{R}_j}), (\tilde{\varepsilon}_i, \tilde{\eta}_i))$ to generate a new attribute key share $\tilde{k}_{attr_i-\mathcal{T}\mathcal{R}_j}$ and send the updated $(Info_{\text{Trap}-\mathcal{T}\mathcal{R}_j}, \tilde{k}_{attr_i-\mathcal{T}\mathcal{R}_j})$ to $\mathcal{T}\mathcal{R}_j$.

VI. SECURITY ANALYSIS

A. Correctness of REBS

Nodes that comply with the redaction policy can find valid hash collisions in **ChCld**, update the verification time t in **TimeUpdate**, and ensure that the chameleon hash value is verified within $t + \Delta t$. An $\mathcal{AVN}$ leaving the P2P network can delegate key distribution authority to another legitimate node in **Delegate**. The correctness analysis details are as follows.

First, given an attribute set ($Attr_{\mathcal{T}\mathcal{R}}, \mathcal{T}\mathcal{R}$) that satisfies the policy $\mathbb{RP} : (A_{\ell \times m}, \rho, \mathcal{T})$, and $(h, t, r, Info_{\text{Trap}}) \leftarrow \text{CHash}(\cdot, Tx, \cdot)$, for $Tx \neq \tilde{T}x$, if the hash tuple $(\tilde{h}, t, \tilde{r}, Info_{\text{Trap}}) \leftarrow \text{CHash}(\cdot, \tilde{T}x, \cdot)$, then $\text{ChVer}(\cdot, \tilde{T}x, \tilde{h}, t, \tilde{r}) = 1$ and $h = \tilde{h}$. Running **AttrKeyGen** and **ChCld** on attributes ($Attr_{\mathcal{T}\mathcal{R}}, \mathcal{T}\mathcal{R}$) outputs a new transaction $\tilde{T}x$ with a new random number $\tilde{r} = (h \cdot \mathcal{H}(\tilde{T}x)^{-t})^{\tilde{d}} \pmod{\varphi(\tilde{n})}$, where $\tilde{e}\tilde{d} \equiv 1 \pmod{\varphi(\tilde{n})}$. The chameleon hash value is computed as $\tilde{h} = \mathcal{H}(\tilde{T}x)^{t\tilde{r}} = h$, meaning $\tilde{r}$ satisfies $\text{ChVer}(\cdot, \tilde{T}x, \tilde{h}, t, \tilde{r}) = 1$ and $h = \tilde{h}$. Thus, nodes meeting the redaction policy can find valid hash collisions.

Next, $\mathcal{T}\mathcal{S}$ runs **TimeUpdate** to update the time t and random number r , updating the hash tuple $(h, t, r, Info_{\text{Trap}})$ to $(h, t + \Delta t, r', Info_{\text{Trap}})$, and generates the signature $Sig_{\mathcal{T}\mathcal{S}}$, where $r' = r \cdot ((\mathcal{H}(Tx)^{\Delta t})^{-1})^{\tilde{d}} \pmod{\varphi(\tilde{n})}$. Verifying $Sig_{\mathcal{T}\mathcal{S}}$ involves checking $e(g, Sig_{\mathcal{T}\mathcal{S}}) = e(g^{\vartheta}, H_{\text{sig}}(Tx, r', ID_{\mathcal{T}\mathcal{S}}))$. The chameleon hash value is calculated as $h' = \mathcal{H}(Tx)^{t+\Delta t, r'} = h$. Hence, r' ensures that $\text{ChVer}(\cdot, Tx, h, t + \Delta t, r') = 1$ and $h = h'$. Therefore, $\mathcal{T}\mathcal{S}$ effectively updates the verification time and ensures that the chameleon hash value passes verification within $t + \Delta t$.

Finally, $\mathcal{AVN}_i$ inputs the attribute name $attr_{\mathcal{AVN}_i} \in Attr_{\mathcal{T}\mathcal{R}}$ into **Delegate** to obtain the new trapdoor auxiliary information $Info_{\text{Trap}-\mathcal{T}\mathcal{R}_j} = (A_{\ell \times m}, \rho), \{\tilde{c}_0, \tilde{c}_{1,l}, \tilde{c}_{2,l}, \tilde{c}_{3,l}\}, \tilde{h}_{\text{Trap}}$. With the cooperation of other $\mathcal{AVN}$ nodes managing attributes in $Attr_{\mathcal{AVN}}$, the new key for $\mathcal{AVN}_i$ should recover the ephemeral trapdoor $Trap$. Assume that the attribute key shares $\{k_{attr_i-\mathcal{T}\mathcal{R}_j}\}_{l \in Attr_{\mathcal{T}\mathcal{R}_j} \setminus attr_{\mathcal{AVN}_i}}$ and attribute values $\{C_{\rho^{-1}(l)}\}_{l \in Attr_{\mathcal{T}\mathcal{R}_j} \setminus attr_{\mathcal{AVN}_i}}$ are known, where $C_{\rho^{-1}(l)} = [c_{1,l} \cdot e(h_{g\mathcal{D}}, c_{3,l}) / e(k_{attr_i-\mathcal{T}\mathcal{R}}, c_{2,l})]$ and $h_{g\mathcal{D}} = H_{g\mathcal{D}}(0, ID_{\mathcal{T}\mathcal{R}})$. The attribute name controlled by $\mathcal{AVN}_i$ is $\rho(l) = attr_{\mathcal{AVN}_i} = attr_{\mathcal{T}\mathcal{R}_j}$, and the corresponding attribute

key share is $k_{\text{attr}_i-\mathcal{T}\mathcal{R}_j} = g^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}} h_{\mathcal{G}\mathcal{D}}^{\tilde{\eta}_i}$. Let $\rho(l) = i$; then, the new trapdoor auxiliary information related to $\text{attr}_{\mathcal{A}\mathcal{V}\mathcal{N}_i}$, $\widetilde{\text{Info}}_{\text{Trap}-\mathcal{T}\mathcal{R}_j}$, is expressed as

$$\begin{cases} \tilde{c}_0 = c_0 = \text{Trap} \cdot e(g, g)^s \\ \tilde{c}_{1,l} = c_{1,l} \cdot \frac{e\left(c_{2,l}, g^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}}\right)}{e(c_{2,l}, \sigma_1)} = e(g, g)^{\lambda_l} \cdot e(g, g)^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}r_l} \\ \tilde{c}_{2,l} = c_{2,l} = g^{r_l} \\ \tilde{c}_{3,l} = c_{3,l} \cdot \frac{(c_{2,l})^{\tilde{\eta}_i}}{\sigma_2} = g^{r_l\tilde{\eta}_i} g^{\theta_l} \\ \tilde{h}_{\text{Trap}} = h_{\text{Trap}} = H(\text{Trap}). \end{cases}$$

Therefore, we have

$$\begin{aligned} C_{\rho^{-1}(i)} &= \frac{\tilde{c}_{1,l} \cdot e(h_{\mathcal{G}\mathcal{D}}, \tilde{c}_{3,l})}{e(k_{\text{attr}_i-\mathcal{T}\mathcal{R}_j}, \tilde{c}_{2,l})} \\ &= \frac{e(g, g)^{\lambda_l} \cdot e(g, g)^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}r_l} \cdot e(h_{\mathcal{G}\mathcal{D}}, g^{r_l\tilde{\eta}_i} g^{\theta_l})}{e\left(g^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}} h_{\mathcal{G}\mathcal{D}}^{\tilde{\eta}_i}, g^{r_l}\right)} \\ &= \frac{e(g, g)^{\lambda_l} \cdot e(h_{\mathcal{G}\mathcal{D}}, g^{\theta_l}) \cdot e(g, g)^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}r_l} \cdot e(h_{\mathcal{G}\mathcal{D}}, g^{r_l\tilde{\eta}_i})}{e(g, g)^{\tilde{e}i\mathcal{T}\mathcal{R}_{ij}r_l} \cdot e(h_{\mathcal{G}\mathcal{D}}, g^{r_l\tilde{\eta}_i})} \\ &= e(g, g)^{\lambda_l} \cdot e(h_{\mathcal{G}\mathcal{D}}, g)^{\theta_l}. \end{aligned}$$

Based on $C_{\rho^{-1}(i)} \cup \{C_{\rho^{-1}(t)}\}_{t \in \text{Attr}_{\mathcal{T}\mathcal{R}_j} \setminus \text{attr}_{\mathcal{A}\mathcal{V}\mathcal{N}_i}}$, calculate

$$\begin{aligned} \prod_{l \in \rho^{-1}(\text{Attr}_{\mathcal{T}\mathcal{R}_j})} (C_l)^{\mu_l} &= \prod_{l \in \rho^{-1}(\text{Attr}_{\mathcal{T}\mathcal{R}_j})} \left(\frac{c_{1,l} \cdot e(h_{\mathcal{G}\mathcal{D}}, c_{3,l})}{e(k_{\text{attr}_{\rho(l)}-\mathcal{T}\mathcal{R}_j}, c_{2,l})} \right)^{\mu_l} \\ &= e(g, g)^s \end{aligned}$$

where μ_l satisfies $\sum_{l \in \rho^{-1}(\text{Attr}_{\mathcal{T}\mathcal{R}_j})} \mu_l \mathbf{A}_l = (1, 0, \dots, 0)$.

Recover the effective ephemeral trapdoor Trap from $c_0 = \text{Trap} \cdot e(g, g)^s$. Given $(\tilde{p}, \tilde{q}) \leftarrow \text{decode}(\text{Trap})$ and the chameleon hash key $k_{\mathcal{T}\mathcal{R}} = (p, q)$, compute d such that $\tilde{e}d \equiv 1 \pmod{\varphi(n\tilde{n})}$, where $\tilde{e}$ is a large prime. Using the ephemeral trapdoor Trap , the hash collision is computed as outlined in first. Thus, $\mathcal{A}\mathcal{V}\mathcal{N}_i$ running *Delegate* can assist $\mathcal{T}\mathcal{R}$ in recovering the ephemeral trapdoor to find the hash collision.

B. Security Proof of REBS

Theorem 1: For any PPT adversary, if the underlying chameleon hash algorithm is strongly indistinguishable, then REBS possesses indistinguishability.

Proof: Assume that a PPT adversary $\mathcal{A}$ can break the indistinguishability of REBS with a nonnegligible advantage. A PPT simulator $\mathcal{S}$ is constructed such that the probability of $\mathcal{A}$ breaking the strong indistinguishability of the CHET is defined as $\text{Adv}_{\mathcal{S}, \text{CHET}}^{\text{SIND}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{SIND}}(\lambda)$. The secure game between $\mathcal{S}$ and $\mathcal{A}$ proceeds as follows.

- 1) *Setup:* $\mathcal{S}$ generates the parameters (excluding those from CHET) by running the relevant algorithms in the Setup phase and sends them to $\mathcal{A}$. $\mathcal{A}$ returns the system parameters gp and a set of public/private key pairs $(pk_{\mathcal{A}\mathcal{V}\mathcal{N}_i}, sk_{\mathcal{A}\mathcal{V}\mathcal{N}_i})$ to $\mathcal{S}$.
- 2) *Query:* $\mathcal{A}$ queries $(Tx, \tilde{Tx}, (\mathbf{A}_{\ell \times m}, \rho, \mathcal{T}))$ via $\mathcal{O}\mathcal{R}$. $\mathcal{S}$ uses $(Tx, \tilde{Tx})$ as input for CHET and outputs (h, t, r) . It generates an ephemeral trapdoor Trap and encrypts it to obtain the ciphertext $\text{Info}_{\text{Trap}}$. $\mathcal{S}$ randomly selects a

bit $b \in \{0, 1\}$. If $b = 0$, $\mathcal{S}$ generates $(\tilde{h}, t, \tilde{r})$ from $\tilde{Tx}$. If $b = 1$, it generates $(h, t, r, \text{Info}_{\text{Trap}})$ from Tx , runs *ChCld*, modifies Tx to $\tilde{Tx}$, and obtains $(h, t, \tilde{r}, \text{Info}_{\text{Trap}})$. $\mathcal{S}$ returns $((\tilde{h}, \tilde{r}), (h, \tilde{r}), t, \text{Info}_{\text{Trap}})$.

- 3) *Guess:* $\mathcal{A}$ outputs a bit $\tilde{b}$, and $\mathcal{S}$ forwards it to CHET.

As $\mathcal{S}$ and $\mathcal{A}$ have the same probability of winning, and on the basis of the one-more RSA inversion assumption, CHET is indistinguishable [42]. Therefore, REBS is indistinguishable, confirming the theorem. ■

Theorem 2: For any PPT adversary, if the underlying chameleon hash algorithm is collision resistant, then REBS has public collision resistance.

Proof: Suppose that there exists a PPT adversary $\mathcal{A}$ that can break the public collision resistance of REBS with a nonnegligible advantage. A PPT simulator $\mathcal{S}$ is constructed, and the probability that $\mathcal{A}$ breaks the collision resistance of the internal CHET is defined as $\text{Adv}_{\mathcal{S}, \text{CHET}}^{\text{PubCR}}(\lambda) = \text{Adv}_{\mathcal{A}, \text{REBS}}^{\text{PubCR}}(\lambda)$. The secure interaction between $\mathcal{S}$ and $\mathcal{A}$ proceeds as follows.

- 1) *Setup:* $\mathcal{S}$ generates all the parameters (excluding those from CHET) by running the relevant algorithms during the Setup phase. It initializes all $\mathcal{A}\mathcal{V}\mathcal{N}_i$ s via *KeyGen* to obtain public and private key pairs $\{pk_{\mathcal{A}\mathcal{V}\mathcal{N}_i}, sk_{\mathcal{A}\mathcal{V}\mathcal{N}_i}\}$.
- 2) *Query:* $\mathcal{A}$ can query $\mathcal{O}\mathcal{R} = \{\text{AttrKeyGen}, \text{AttrKeyGen}, \text{ChCld}, \text{ChCld}\}$. For the input $(ID_{\mathcal{T}\mathcal{R}}, (\text{attr}_{\mathcal{T}\mathcal{R}_i}, t_{\mathcal{T}\mathcal{R}_i}))$: $\mathcal{A}$ queries the attribute key generation oracle with identity $ID_{\mathcal{T}\mathcal{R}}$ and attribute $(\text{attr}_{\mathcal{T}\mathcal{R}_i}, t_{\mathcal{T}\mathcal{R}_i})$. and *KeyGen* outputs $(k_{\mathcal{T}\mathcal{R}}, \text{Sig}_{\mathcal{A}\mathcal{M}\mathcal{C}})$. For each $(\text{attr}_{\mathcal{T}\mathcal{R}_i}, t_{\mathcal{T}\mathcal{R}_i}) \in \{\text{Attr}_{\mathcal{T}\mathcal{R}}, \mathcal{T}_{\mathcal{T}\mathcal{R}}\}$, the *AttrKeyGen* outputs $k_{\text{attr}_i-\mathcal{T}\mathcal{R}}$. Since $\mathcal{A}$ is the adversary, $\mathcal{S}$ stores keys $k_{\mathcal{T}\mathcal{R}}, \{k_{\text{attr}_i-\mathcal{T}\mathcal{R}}\}_{i \in \text{Attr}_{\mathcal{T}\mathcal{R}}}$. For *ChCld*: $\mathcal{A}$ queries the transaction Tx , hash value h , time t , random number r , auxiliary information $\text{Info}_{\text{Trap}}$, and new transaction $\tilde{Tx}$. The tuple $(Tx, h, t, r, \tilde{Tx})$ is sent to CHET, which returns a new random number $\tilde{r}$ to $\mathcal{A}$.
- 3) *Output:* $\mathcal{A}$ outputs $(Tx_0, r_0, \tilde{Tx}_0, \tilde{r}_0, t_0, h_0, \text{Info}_{\text{Trap}_0})$ to $\mathcal{S}$, which forwards it to CHET.

From the above process, for any message $\{Tx, \tilde{Tx}_0\}$ not queried from oracle $\mathcal{O}\mathcal{R}$, $\mathcal{S}$ can find a valid collision tuple $(Tx_0, r_0, \tilde{Tx}_0, \tilde{r}_0, t_0, h_0, \text{Info}_{\text{Trap}_0})$ with the same probability as $\mathcal{A}$. This implies that $\mathcal{S}$ and $\mathcal{A}$ have the same chance of success. On the basis of the RSA assumption, if there is no valid Trap_0 , $\mathcal{S}$ cannot compute $\tilde{r}_0$. Therefore, REBS has public collision resistance, and the theorem is proven. ■

Theorem 3: For any PPT adversary, if the underlying MA-ABE scheme provides static security (IND-CCA2) in the CCA setting, the chameleon hash algorithm is collision resistant, and the digital signature (DS) scheme is unforgeable under adaptive chosen message attacks (EUF-CMA), then REBS exhibits private collision resistance.

Proof: Let $\mathcal{C}$, $\mathcal{M}$, and $\mathcal{D}$ represent adversaries targeting the private collision resistance of CHET, IND-CCA2 security of MA-ABE, and EUF-CMA security of DS, respectively. Let $\text{Pr}[G_i]$ denote the probability that the adversary succeeds in game Game_i , and let q be the number of *ChCld* queries. The games are as follows.

- 1) *Game₀*: This corresponds to the private collision resistance experiment $\text{PriCollRes}^{\text{REBS}}_{\mathcal{A}}(\lambda)$.
- 2) *Game₁*: The adversary $\mathcal{A}$ lacks sufficient attributes for the k th query $(Tx_0, h_0, t_0, r_0, \text{Info}_{\text{Trap}_0}, \widetilde{Tx}_0)$, where Tx_0 and $\widetilde{Tx}_0$ are unmodified transactions. The oracle $\mathcal{OR}$ runs **ChCld** as CHET.ChCld. If $\mathcal{A}$ discovers a valid collision, the game ends. Otherwise, it mirrors *Game₀*. The probability of winning remains the same, $\Pr[G_1] = \Pr[G_0] \cdot (1/q)$.
- 3) *Game₂*: The ephemeral trapdoor Trap_0 is used directly for collisions without decrypting $\text{Info}_{\text{Trap}_0}$. The rest are identical to *Game₁*, so $\Pr[G_2] = \Pr[G_1]$.
- 4) *Game₃*: In oracle $\mathcal{OR}$, when **ChCld** queries CHET.ChCld, $\mathcal{M}$ encrypts 0 instead of the real Trap_0 . The game remains indistinguishable from *Game₂* because of the IND-CCA2 security of MA-ABE, meaning $|\Pr[G_3] - \Pr[G_2]| \leq \text{Adv}^{\text{IND-CCA2}}_{\mathcal{M}, \text{MA-ABE}}(\lambda)$.
- 5) *Game₄*: If $\mathcal{A}$ finds a valid collision, the game terminates and returns the information $(Tx_0, r_0, \widetilde{Tx}_0, \tilde{r}_0, t_0, h_0, \text{Info}_{\text{Trap}_0})$ to $\mathcal{C}$, who attempts to break the CHET's private collision resistance. The probability remains the same as in *Game₃*, $\Pr[G_4] = \Pr[G_3]$, with $\Pr[G_4] \leq \text{Adv}^{\text{PriCR}}_{\mathcal{C}, \text{CHET}}(\lambda)$.
- 6) *Game₅*: Honest attribute verification nodes with initial identifiers $ID_{\mathcal{TR}}$ and chameleon hash keys $k_{\mathcal{TR}}$ are represented by $\mathcal{D}$, which can generate keys for new transaction editors. The rest of *Game₅* is identical to *Game₄*, with $\Pr[G_5] = \Pr[G_4]$ and $\Pr[G_5] \leq \text{Adv}^{\text{EUF-CMA}}_{\mathcal{D}, \text{DS}}(\lambda)$.

In *Game₀*, the adversary's success probability is $\Pr[G_0] \leq q \cdot (\text{Adv}^{\text{PriCR}}_{\mathcal{C}, \text{CHET}}(\lambda) + \text{Adv}^{\text{IND-CCA2}}_{\mathcal{M}, \text{MA-ABE}}(\lambda) + \text{Adv}^{\text{EUF-CMA}}_{\mathcal{D}, \text{DS}}(\lambda))$. Since each of these terms is negligible, $\Pr[G_0] \leq f(\lambda)$, and thus, the adversary's probability of success is negligible. Therefore, REBS exhibits private collision resistance, confirming the theorem.

Furthermore, since REBS is resistant to key compromise attacks, it withstands both public and private collisions. Even with a message collision, key leakage is less severe than public collision resistance, preventing any PPT adversary from extracting the key to find a hash collision [43]. ■

Theorem 4: If assumptions 1, 2, 3 and 4⁷ hold, then REBS exhibits the partial policy hiding property.

On the basis of complexity assumptions 1, 2, 3 and 4, we use a series of game sequences to prove that REBS exhibits partial strategy hiding. Let q represent the number of key queries made by the adversary $\mathcal{A}$. For k ranging from 1 to q , we define the following game sequences.

- 1) *Game_{Real}*: The ciphertext and keys are identical to those of the original scheme.
- 2) *Game_{Real'}*: The random oracle maps the redactor's identifier $ID_{\mathcal{TR}}$ randomly to $\mathbb{G}_{p_1}$, with all other aspects the same as in *Game_{Real}*.

- 3) *Game₀*: The challenge ciphertext is a semifunctional ciphertext of the *SemiFuncCip-Info_{Trap}* type, with all other aspects the same as in *Game_{Real'}*.
- 4) *Game_{k,1}*: The first $k-1$ keys received are of the *SemiFuncKey- $K_{\text{Attr-}\mathcal{TR}-2}$* type, the k th key is of the *SemiFuncKey- $K_{\text{Attr-}\mathcal{TR}-1}$* type, and all other aspects are the same as in *Game₀*.
- 5) *Game_{k,2}*: The first k keys received are of the *SemiFuncKey- $K_{\text{Attr-}\mathcal{TR}-2}$* type, with all other aspects the same as in *Game₀*. In *Game_{k,2}*, all keys are of the *SemiFuncKey- $K_{\text{Attr-}\mathcal{TR}-2}$* type.
- 6) *Game_{Final₀}*: The ciphertext is a semifunctional ciphertext *SemiFuncCip-Info_{Trap}* of a random message Tx , and all keys are of the *SemiFuncKey- $K_{\text{Attr-}\mathcal{TR}-2}$* type. $\mathcal{A}$ has no advantage in this game, i.e., $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_0}(\lambda) \leq f(\lambda)$.
- 7) *Game_{Final₁}*: Part of the challenge ciphertext information $c_{1,l}$ is randomly chosen from $\mathbb{G}_{p_1 p_2 p_3}$. All other aspects are the same as in *Game_{Final₀}*. $\mathcal{A}$ has no advantage in this game, i.e., $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_1}(\lambda) \leq f(\lambda)$.

The relevant lemma (the derivation path in Fig. 2) is as follows.

Lemma 1: Assume the existence of a PPT adversary $\mathcal{A}$, such that $\epsilon(\lambda) = \text{Adv}^{\text{Game}}_{\mathcal{A}}(\lambda) - \text{Adv}^{\text{Game}'}_{\mathcal{A}}(\lambda)$. If REBS satisfies this assumption, then $\epsilon(\lambda) \leq f(\lambda)$, meaning that *Game* and *Game'* are computationally indistinguishable, where *Game*, *Game'* $\in \{\text{Game}_{\text{Real}}, \text{Game}_{\text{Real}'}, \text{Game}_0, \text{Game}_{k,1}, \text{Game}_{k,2}, \text{Game}_{\text{Final}_0}, \text{Game}_{\text{Final}_1}\}$.

Proof: Suppose that the probabilities $\mathcal{A}$ wins in *Game_{Final₀}* and *Game_{Final₁}* are $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_0}(\lambda)$ and $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_1}(\lambda)$, respectively. In *Game_{Final₁}*, part of the challenge ciphertext $c_{1,l}$ belongs to $\mathbb{G}_{p_1 p_2 p_3}$, whereas the other parts are identical to those in *Game_{Final₀}*. Part of the initial challenge ciphertext is $\hat{c}_{1,l} = e(g_1, g_1)^{\lambda_l} e(g_1, g_1)^{\epsilon_{\rho(0)} \epsilon_{\rho(0)} \epsilon_l}$, and from this expression, it is clear that $\hat{c}_{1,l} \in \mathbb{G}_T$. Since REBS satisfies Assumption 4, *Game_{Final₀}* and *Game_{Final₁}* are computationally indistinguishable. Thus, $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_0}(\lambda) - \text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_1}(\lambda) \leq f(\lambda)$, confirming Lemma 1. The ciphertext and key structure of REBS are similar to those in [40], and the semifunctional ciphertext and key generated on the basis of this structure are also similar. Therefore, the proofs of the preceding steps are omitted here; interested readers can refer to [40]. When $k = 0$, *Game₀* is identical to *Game_{k,2}*. When $k = q$, *Game_{k,1}* and *Game_{k,2}* are identical to *Game_{q,1}* and *Game_{q,2}*, respectively. From the derivation path in Fig. 2, it is evident that *Game_{Real}* and *Game_{Final₁}* are computationally indistinguishable. Given the indistinguishability of *Game_{Real}* and *Game_{Final₁}*, the probability that $\mathcal{A}$ wins in *Game_{Final₁}* is $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Final}_1}(\lambda) \leq f(\lambda)$. Therefore infer that the probability of $\mathcal{A}$ winning in *Game_{Real}* is $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Real}}(\lambda) \leq f(\lambda)$, meaning that $\text{Adv}^{\text{Game}}_{\mathcal{A}, \text{Real}}$ is negligible, thus proving Theorem 4. ■

VII. PERFORMANCE EVALUATION

A. Computational Overhead of REBS

This section compares the computational overhead of representative redactable blockchain schemes (PCH [17],

⁷Lewko and Waters [44] demonstrated the security of assumptions 1, 2, 3, and Caro et al. [45] proved the security of assumption 4.

TABLE II
COMPARISON OF COMPUTATIONAL OVERHEAD OF SEVERAL REPRESENTATIVE REDACTABLE BLOCKCHAIN SCHEMES

Schemes	KeyGen	Hash	Verify	Adapt
PCH [17] (2019)	$(15 + 9s)E + (9 + 6s)M + (6 + 6s)H$	$(7 + 6l + 9s)E + (4 + 3l + 6sl)M + (3 + 6l + 6s)H + SE$	$2E + 2M + 2BP$	$(11 + 6k + 6l + 9sl)E + (10 + 6k + 6l + 6sl)M + 6BP + (3 + 6l + 6s)H + SE$
DPCH [25] (2022)	$4sE + 2sM + 2BP + 3sH$	$(3 + 6l)E + (3 + 2l)M + (1 + 2l)BP + (2 + 2l + 2s)H + SE$	$2E + 2M + 2BP$	$(3 + k + 6l)E + (3 + 4k + 2l)M + (1 + 3k + 2l)BP + (4 + 2l + 2s)H + SE$
PCHA [19] (2023)	$(15 + 9s)E + (9 + 6s)M + (6 + 6s)H$	$(23 + 6l + 9s)E + (4 + 3l + 6sl)M + 3BP + (5 + 12l + 6s)H + SE$	$4E + 3M + 3BP + 2H$	$(32 + 7k + 6l + 9s)E + (17 + 8k + 3l + 6sl)M + 9BP + (11 + k + 13l + 6s)H + 2SE$
PRHBS [33] (2024)	$(3 + 4s)E + (1 + 2s)M$	$(7 + 7l)E + (3 + 2l)M + 2BP + 3H$	$2E + M + BP$	$(1 + k)E + (1 + 3k)M + (1 + 3k)BP + H + SE$
Ours	$2sE + sM + 2BP + (1 + s)H$	$(2 + 5l)E + (2 + 2l)M + (1 + 2l)BP + 2H$	$E + M + BP$	$(1 + k)E + (1 + 2k)M + 2kBP + 2H$

- E and M denote exponentiation and multiplication operations on the group, respectively; BP represents the bilinear mapping operation; H stands for the hash operation; SE signifies the symmetric encryption operation.

- l denotes the number of attributes in the access policy; s represents the number of attributes owned by the user; k is the number of attributes in the decryption key that satisfy the access policy, where $k \leq l$.

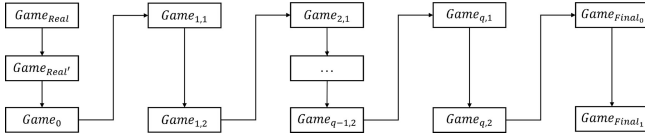


Fig. 2. Derivation path from $\text{Game}_{\text{Real}}$ to $\text{Game}_{\text{Final}_i}$

DPCH [25], PCHA [19], and PRHBS [33]) to assess the computational efficiency of REBS. These schemes, like ours, use attribute-based redaction permissions and CHFs to support transaction redaction in blockchain.

Table II presents the theoretical computational overhead of the comparison schemes, including **KeyGen**, **Hash(CHash)**, **Verify(ChVer)**, and **Adapt (ChCld)**. Generally, the overhead increases with the number of attributes, but REBS has a lower time complexity than the other methods do. Additionally, REBS supports dynamic node updates (i.e., **Delegate**) and verification time updates with time (i.e., **Update**) complexities of $6kE + 4kM + 7kBP + 2kH$ and $3E + 2M + 2BP + H$, respectively. In summary, REBS achieves redaction policy hiding with reduced computational overhead, supports dynamic node updates and verification time updates, and prevents the misuse of redaction permissions.

To further evaluate the feasibility and performance of REBS, we used Python along with Charm-Crypto 0.5, which relies on the PBC, GMP, and OpenSSL libraries for algebraic operations and communication setups.⁸ The experiments were conducted on an Intel Core i5-8250U 1.80 GHz CPU with 8 GB RAM. We implemented the SS512 (supersingular 512-bit) curve for bilinear pairing and RSA with 1024-bit keys, along with SHA256 for hash functions and pseudorandom number generation. The experiments were repeated 100 times, and the average results were used for final evaluation to ensure objectivity.

As shown in Fig. 3, as the number of required attributes in the redaction policies increased from 10 to 100, the runtime of most algorithms increased at varying rates. Compared with other schemes, our approach has significant performance

advantages. Notably, as seen in Fig. 3(c) and (f), the computational overhead for **Verify** and **Update** algorithms remains constant, unaffected by the number of required attributes. Fig. 3(e) and (f) illustrate that the **Delegate** and **Update** algorithms add functionality without significantly increasing resource consumption. Specifically, the **Delegate** algorithm's computational overhead scales linearly with the number of attributes, taking only 0.5 s for 100 attributes. The **Update** algorithm's overhead is constant, with a consistent runtime of 0.0057 s.

Fig. 4 shows the total computational overhead of the comparison schemes. Without dynamic node exit and verification time updates, as in [25] and [33], REBS achieves 0.130 s less runtime than does [33] and 0.801 s less runtime than does [25] when up to 100 attributes are considered. Fig. 5 depicts the computational overhead of REBS as the number of attributes increases from 10 to 60. Specifically, when $l = 40$, the time bounds for 20 experimental runs are 0.608 s (upper bound) and 0.320 s (lower bound), with an average runtime of 0.432 s. Fig. 6 shows the computational overhead distributions for the algorithms in this scheme. Specifically, the computational overhead of REBS mainly consists of the runtimes of algorithms **KeyGen**, **Hash(CHash)**, **Adapt(ChCld)**, and **Delegate**, where different selections of parameters l , k , and s influence the results. For example, when $s = 20$ and $k = 15$, as l increases from 20 to 30, the time overhead proportions for algorithms **Hash** and **Delegate** change from 36% and 46% to 46% and 39%, respectively. Therefore, in practical applications, adjusting these parameters can further optimize REBS performance. Overall, the experimental data demonstrate that REBS delivers high performance while expanding functionality.

B. Communication Overhead of REBS

This section compares the communication overhead of several redactable blockchain schemes (PCH [17], DPCH [25], PCHA [19], and PRHBS [33]) to evaluate the communication efficiency of REBS. Theoretical analysis examines the relationship between the number of attributes and communication overhead. Specifically, for the chameleon hash algorithm in REBS, communication overhead is measured on the basis of three components: 1) the attribute key $K_{Attr-\mathcal{T}(\mathcal{R})}$; 2) the

⁸<https://github.com/JHUISI/charm>; <http://crypto.stanford.edu/pbc/>; <https://gmplib.org/>; <https://github.com/openssl/openssl>.

TABLE III
COMPARISON OF COMMUNICATION OVERHEAD OF SEVERAL REPRESENTATIVE REDACTABLE BLOCKCHAIN SCHEMES

Schemes	Attribute Key Size	Hash Tuple Size	Ciphertext Size
PCH [17]	$(1 + 3k) \mathbb{G} + 3 \mathbb{G}_{p_1} $	$3 \mathbb{Z}_N^* $	$3l \mathbb{G} + 3 \mathbb{G}_{p_1} + 2 \mathbb{G}_T + \mathbb{Z}_N^* $
DPCH [25]	$2k \mathbb{G} $	$6 \mathbb{Z}_N^* $	$3l \mathbb{G} + (1 + l) \mathbb{G}_T + \mathbb{Z}_N^* $
PCHA [19]	$(9 + 3k) \mathbb{G} $	$5 \mathbb{Z}_N^* $	$(10 + 3l) \mathbb{G} + 2 \mathbb{Z}_N^* $
PRHBS [33]	$(2 + 2k) \mathbb{G} $	$(4 + k) \mathbb{Z}_N^* $	$(1 + 3l) \mathbb{G} + \mathbb{G}_T $
Ours	$k \mathbb{G} $	$3 \mathbb{Z}_N^* $	$2l \mathbb{G} + (1 + l) \mathbb{G}_T + \mathbb{Z}_N^* $

$|\mathbb{G}|$, $|\mathbb{G}_{p_1}|$, $|\mathbb{G}_T|$, and $|\mathbb{Z}_N^*|$ denote the bit lengths of the elements in the groups $\mathbb{G}$, $\mathbb{G}_{p_1}$, $\mathbb{G}_T$, and $\mathbb{Z}_N^*$, respectively.

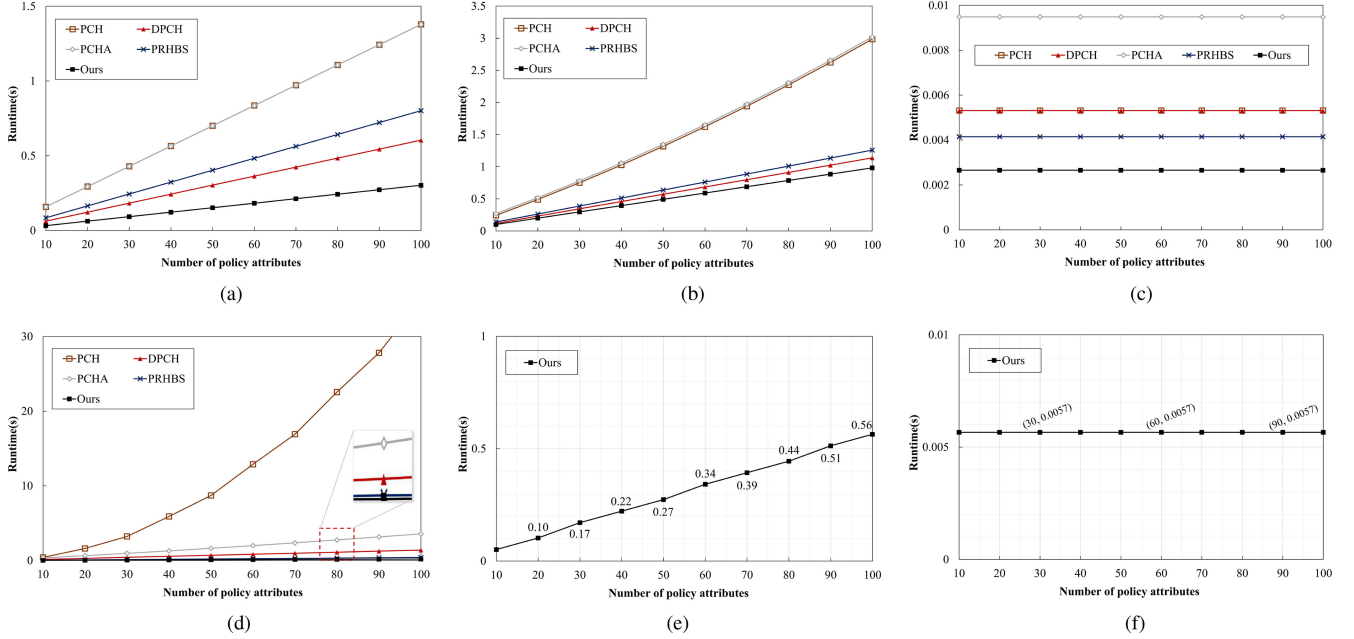


Fig. 3. Comparison of computational overhead. (a) Running time for *KeyGen*. (b) Running time for *Hash*. (c) Running time for *Verify*. (d) Running time for *Adapt*. (e) Running time for *Delegate*. (f) Running time for *Update*.

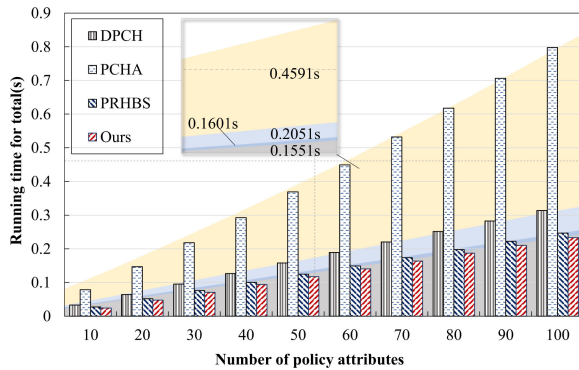


Fig. 4. Comparison of total computational overhead.

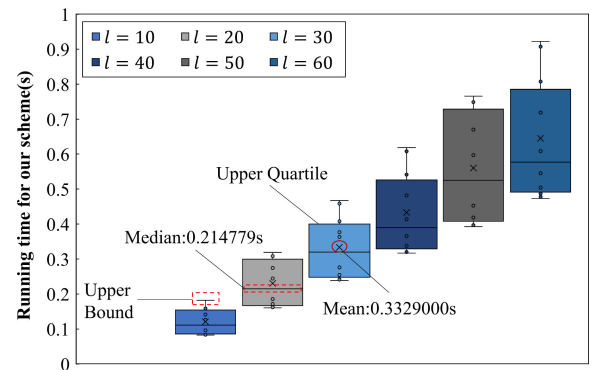


Fig. 5. Computational overhead of REBS in actual environments.

hash tuple (h, t, r) ; and 3) the ciphertext (trapdoor auxiliary information) *InfoTrap*. As shown in Table III, the sizes of the attribute keys and ciphertexts in each scheme increase with the number of attributes satisfying the redactable policy. Except for [33], the size of the hash tuples in the other schemes remains constant regardless of the number of attributes. Our scheme's attribute key and hash tuple sizes are smaller than those of other schemes, and the ciphertext size is lower than that in [25]. Therefore, REBS demonstrates practical efficiency in both communication and storage.

To evaluate the communication efficiency of REBS in practice, we conducted experiments with the following parameter sizes: 512-bit elements in $\mathbb{G}$ and $\mathbb{G}_{p_1}$, 1024-bit elements in $\mathbb{G}_T$, and 1024-bit elements in $\mathbb{Z}_N^*$. The total number of attributes in the policy matrix (i.e., the number of rows) l ranged from 10 to 50, with the number of attributes in the authorization set k satisfying $k \leq l$.

Fig. 7 presents experimental data on the communication overhead of various schemes. REBS outperforms [25] and is comparable to [17], [19], and [33]. Our scheme employs

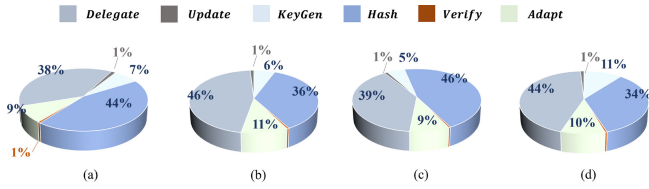


Fig. 6. Computational overhead of algorithms involved in REBS. (a) $l = 20, k = 10, s = 10$. (b) $l = 20, k = 15, s = 10$. (c) $l = 30, k = 15, s = 10$. (d) $l = 20, k = 15, s = 20$.

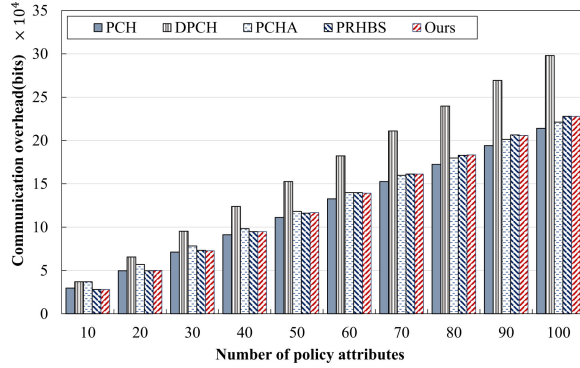


Fig. 7. Comparison of communication overhead.

a CHF with one ephemeral trapdoor resistant to key leakage attacks. To allow nodes that meet the policy to obtain ephemeral trapdoors and edit energy transactions, REBS requires attribute encryption for the trapdoors, slightly increasing the communication overhead. With ten attributes ($l = 10$), REBS reduces communication overheads by 23.6% compared with [25] and 23.5% compared with [19]. The communication overhead is approximately 28 160 bits, similar to [17] and [33]. When the number of attributes increases to 50 ($l = 50$), REBS reduces communication overheads by 23.5% compared with [25]. For 100 attributes, REBS's communication overhead is only 227 840 bits, which is comparable to [17], [19], and [33] and significantly lower than [25]. Fig. 8 illustrates the communication overhead distribution across three components in REBS: attribute keys, hash tuples, and ciphertext (trapdoor auxiliary information). With 50 attributes, the proportions of communication overhead for attribute keys, hash tuples, and ciphertext are 7.1%, 2.7%, and 90.3%, respectively. As the number of attributes increases, the ciphertext size consistently accounts for over 80% of the total communication overhead in REBS. In summary, REBS supports system security while maintaining lower communication overhead.

C. REBS in IoT Applications

REBS exhibits notable compatibility in the IoT ecosystem.

- 1) *Data Consistency and Security*: In the IoT, distributed devices yield copious sensitive data. REBS employs a redactable blockchain for storage, ensuring consistency and privacy via attribute-based policy hiding and authorizable verification, curbing leakage and misuse risks.
- 2) *Flexible Access Control*: REBS empowers dynamic permission management among IoT devices through controlled multiauthority key generation and redactor

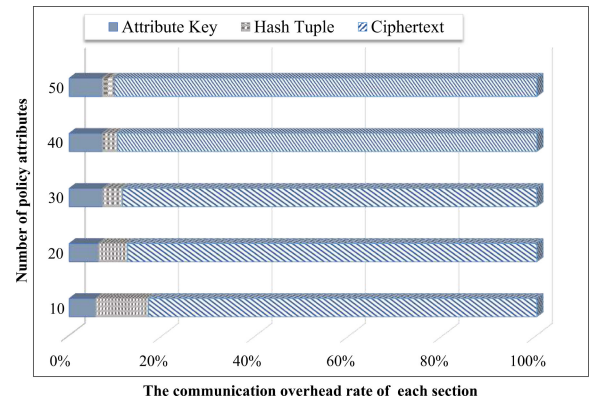


Fig. 8. Communication overhead rate of each section in REBS.

attribute verification, affording precise control of security and redaction rights per node.

- 3) *Decentralized and Autonomous Transactions*: REBS backs decentralized energy trading, permitting secure and clear data exchange and transactions between IoT devices and sans centralized trust.
- 4) *Scalability and Performance*: REBS features efficient data validation and access control, meeting the high processing needs of large-scale IoT networks and ensuring robust performance and scalability.

VIII. CONCLUSION AND FUTURE WORK

We propose a REBS to address issues of trapdoor security risk and indefinite redactions in distributed energy trading management. Our solution includes an attribute-based redaction policy that integrates attribute verification and transaction-level content redaction by using a chameleon hash with ephemeral trapdoors. To protect node attribute values and mitigate privacy concerns for redaction nodes, we implement a partially attribute-based policy hiding mechanism. Compared with existing redactable blockchain schemes, REBS has an extended functionality with time-limited redaction and authorizable verification, increasing its practicality. Despite progress, REBS has the following security risk and limits in practice. It hides only node attribute values, leaving names visible. In high security, this risks privacy leaks. Furthermore, REBS uses re-encryption for node flexibility, but it adds overhead, making resource-limited devices bad for verification.

Future work will focus on overcoming these limitations. Specifically, we intend to harness hidden vector encryption to obscure attribute-related information during the encryption process. By employing distinct vectors to regulate access for diverse nodes, comprehensive policy hiding can be attained. We also envisage integrating puncturable encryption to optimize the existing mechanism for attribute verification nodes, thereby enhancing both the efficiency and flexibility of managing such nodes. Moreover, efforts will be dedicated to refining the design of the redactable blockchain to ensure the traceability of redaction operations. This, in turn, facilitates the identification of malicious nodes and precludes the misuse of redaction permission.

REFERENCES

- [1] W. Tushar et al., "A motivational game-theoretic approach for peer-to-peer energy trading in the smart grid," *Appl. Energy*, vol. 243, pp. 10–20, Jun. 2019.
- [2] M. Li, D. Hu, C. Lal, M. Conti, and Z. Zhang, "Blockchain-enabled secure energy trading with verifiable fairness in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 16, no. 10, pp. 6564–6574, Oct. 2020.
- [3] Y. Wang et al., "SPDS: A secure and auditable private data sharing scheme for smart grid based on blockchain," *IEEE Trans. Ind. Informat.*, vol. 17, no. 11, pp. 7688–7699, Nov. 2021.
- [4] B. Wang, L. Xu, and J. Wang, "A privacy-preserving trading strategy for blockchain-based P2P electricity transactions," *Appl. Energy*, vol. 335, Apr. 2023, Art. no. 120664.
- [5] S. Sayeed, H. Marco-Gisbert, and T. Caira, "Smart contract: Attacks and protections," *IEEE Access*, vol. 8, pp. 24416–24427, 2020.
- [6] M. I. Mehar et al., "Understanding a revolutionary and flawed grand experiment in blockchain: The DAO attack," *J. Cases Inf. Technol.*, vol. 21, no. 1, pp. 19–32, 2019.
- [7] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," in *Proc. 6th Int. Conf. Principles Security Trust*, 2017, pp. 164–186.
- [8] Parliament and the Council of the European Union, "General data protection regulation (GDPR)," Official Journal of the European Union, 2016. Accessed: Apr. 18, 2024. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX>
- [9] National People's Congress, "Personal information protection law," 2021. Accessed: Apr. 18, 2024. [Online]. Available: <http://www.npc.gov.cn/npc/c30834/202108/a8c4e3672c74491a80b53a172bb753fe.shtml>
- [10] G. Ateniese, B. Magri, D. Venturi, and E. R. Andrade, "Redactable blockchain—or—rewriting history in bitcoin and friends," in *Proc. IEEE Eur. Symp. Security Privacy, EuroSP*, 2017, pp. 111–126.
- [11] I. Puddu, A. Dmitrienko, and S. Capkun, "μchain: How to forget without hard forks," IACR Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 106/2017, 2017. [Online]. Available: <http://eprint.iacr.org/2017/106>
- [12] D. Deuber, B. Magri, and S. A. K. Thyagarajan, "Redactable blockchain in the permissionless setting," in *Proc. IEEE Symp. Security Privacy (SP)*, 2019, pp. 124–138.
- [13] Y. Ren, X. Cai, and M. Hu, "Privacy-preserving redactable blockchain for Internet of Things," *Secur. Commun. Netw.*, vol. 6, pp. 1–12, Jan. 2021.
- [14] M. Florian, S. A. Henningsen, S. Beaucamp, and B. Scheuermann, "Erasing data from blockchain nodes," in *Proc. IEEE Eur. Symp. Security Privacy Workshops, EuroSP*, 2019, pp. 367–376.
- [15] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, "Reparo: Publicly verifiable layer to repair blockchains," in *Proc. 25th Int. Conf. Financ. Cryptogr. Data Security*, 2021, pp. 37–56.
- [16] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, "Escaping from consensus: Instantly redactable blockchain protocols in permissionless setting," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3699–3715, Sep./Oct. 2023.
- [17] D. Derler, K. Samelin, D. Slamanig, and C. Striecks, "Fine-grained and controlled rewriting in blockchains: Chameleon-hashing gone attribute-based," in *Proc. 26th Annu. Netw. Distrib. Syst. Security Symp.*, 2019, pp. 1–15.
- [18] Y. Tian, N. Li, Y. Li, P. Szalachowski, and J. Zhou, "Policy-based chameleon hash for blockchain rewriting with black-box accountability," in *Proc. Annu. Comput. Security Appl. Conf.*, 2020, pp. 813–828.
- [19] S. Xu, X. Huang, J. Yuan, Y. Li, and R. H. Deng, "Accountable and fine-grained controllable rewriting in blockchains," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 101–116, 2023.
- [20] G. Panwar, R. Vishwanathan, and S. Misra, "Retrace: Revocable and traceable blockchain rewrites using attribute-based cryptosystems," in *Proc. 26th ACM Symp. Access Control Models Technol.*, 2021, pp. 103–114.
- [21] S. Chen, J. Li, Y. Zhang, and J. Han, "Efficient revocable attribute-based encryption with verifiable data integrity," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10441–10451, Mar. 2024.
- [22] Y. Jia, S. Sun, Y. Zhang, Z. Liu, and D. Gu, "Redactable blockchain supporting supervision and self-management," in *Proc. ACM Asia Conf. Comput. Commun. Security*, 2021, pp. 844–858.
- [23] W. Shao, J. Wang, L. Wang, C. Jia, S. Xu, and S. Zhang, "Auditable blockchain rewriting in permissioned setting with mandatory revocability for IoT," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21322–21336, Dec. 2023.
- [24] S. Xu, J. Ning, J. Ma, X. Huang, and R. H. Deng, "K-time modifiable and epoch-based redactable blockchain," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4507–4520, 2021.
- [25] J. Ma, S. Xu, J. Ning, X. Huang, and R. H. Deng, "Redactable blockchain in decentralized setting," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1227–1242, 2022.
- [26] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, "Secure redactable blockchain with dynamic support," *IEEE Trans. Dependable Security Comput.*, vol. 21, no. 2, pp. 717–731, 2024.
- [27] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, and X. Du, "Scalable and redactable blockchain with update and anonymity," *Inf. Sci.*, vol. 546, pp. 25–41, Feb. 2021.
- [28] M. Jia et al., "Redactable blockchain from decentralized chameleon hash functions," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2771–2783, 2022.
- [29] J. Shen, X. Chen, Z. Liu, and W. Susilo, "Verifiable and redactable blockchains with fully editing operations," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 3787–3802, 2023.
- [30] J. Li, H. Ma, J. Wang, Z. Song, W. Xu, and R. Zhang, "Wolverine: A scalable and transaction-consistent redactable permissionless blockchain," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1653–1666, 2023.
- [31] W. Dai et al., "PRBFPT: A practical redactable blockchain framework with a public trapdoor," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2425–2437, 2024.
- [32] Y. Zhang, Z. Ma, S. Luo, and P. Duan, "Dynamic trust-based redactable blockchain supporting update and traceability," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 821–834, 2024.
- [33] S. Xu, J. Ning, X. Li, J. Yuan, X. Huang, and R. H. Deng, "A privacy-preserving and redactable healthcare blockchain system," *IEEE Trans. Serv. Comput.*, vol. 17, no. 2, pp. 364–377, Mar./Apr. 2024.
- [34] K. Ashritha, M. Sindhu, and K. V. Lakshmy, "Redactable blockchain using enhanced chameleon hash function," in *Proc. 5th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, 2019, pp. 323–328.
- [35] P. Li, H. Xu, T. Ma, and Y. Mu, "Research on fault-correcting blockchain technology," *J. Cryptol. Res.*, vol. 5, no. 5, pp. 501–509, 2018.
- [36] J. Camenisch, D. Derler, S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-Hashes with ephemeral trapdoors—And applications to invisible sanitizable signatures," in *Proc. Int. Conf. Pract. Theory Public-Key Cryptogr.*, 2017, pp. 152–182.
- [37] S. Agrawal and M. Chase, "FAME: Fast attribute-based message encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security, CCS*, 2017, pp. 665–682.
- [38] J. Lai, R. H. Deng, and Y. Li, "Expressive CP-ABE with partially hidden access structures," in *Proc. 7th ACM Symp. Inf. Comput. Commun. Security*, 2012, pp. 18–19.
- [39] Z. Liu, Z. Cao, and D. S. Wong, "Efficient generation of linear secret sharing scheme matrices from threshold access trees," *Cryptology ePrint Archive*, IACR, Bellevue, WA, USA, Rep. 374/2010, 2010. [Online]. Available: <https://eprint.iacr.org/2010/374>
- [40] A. B. Lewko and B. Waters, "Decentralizing attribute-based encryption," in *Proc. 30th Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2011, pp. 568–588.
- [41] J. Katz, A. Sahai, and B. Waters, "Predicate encryption supporting disjunctions, polynomial equations, and inner products," in *Proc. 27th Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2008, pp. 146–162.
- [42] M. Bellare, C. Namprepmpre, D. Pointcheval, and M. Semanko, "The one-more-RSA-inversion problems and the security of Chaum's blind signature scheme," *J. Cryptol.*, vol. 16, no. 3, pp. 185–215, 2003.
- [43] G. Ateniese and B. de Medeiros, "On the key exposure problem in chameleon hashes," in *Proc. 4th Int. Conf. Secur. Commun. Netw.*, 2004, pp. 165–179.
- [44] A. B. Lewko and B. Waters, "New techniques for dual system encryption and fully secure HIBE with short ciphertexts," in *Proc. 7th Theory Cryptogr. Conf.*, 2010, pp. 455–479.
- [45] A. D. Caro, V. Iovino, and G. Persiano, "Fully secure anonymous HIBE and secret-key anonymous IBE with short ciphertexts," in *Proc. 4th Int. Conf. Pairing-Based Cryptogr. Pairing*, 2010, pp. 347–366.



Jingting Xue received the B.Sc. and Ph.D. degrees from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2014 and 2020, respectively.

She is currently an Associate Professor with the School of Computer Science, Southwest Petroleum University, Chengdu. In June 2023, she joined the School of Computer Science and Engineering, UESTC, and the Science and Technology on Communication Security Laboratory, Institute of Southwestern Communication, Mianyang, China, as a Postdoctoral Research Fellow. During the Ph.D. studies, she was a visiting Ph.D. student with Nanyang Technological University, Singapore, in 2019, where she conducted research for one year. Her research interests include applied cryptography, cloud storage, and blockchain technology.



Wenzheng Zhang received the B.E. degree in computer systems organization from Southwest Communication Research Institute, Chengdu, China.

He is currently a Senior Engineer with the Science and Technology on Communication Security Laboratory, Chengdu. His research interests include cryptographic theory and technologies.



Liang Liu received the B.Sc. degree from Xihua University, Chengdu, China, in 2022. He is currently pursuing the M.S. degree in computer science with the School of Computer Science and Software Engineering, Southwest Petroleum University, Chengdu.

His research focuses on blockchain applications and secure multiparty computation systems.



Xiaojun Zhang received the Ph.D. degree in information security from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2015.

He is currently an Associate Professor with the School of Computer Science, Southwest Petroleum University, Chengdu. From 2016 to 2019, he was a Postdoctoral Research Fellow with UESTC, and from 2020 to 2022, he continued his Postdoctoral Research with the School of Cyber Science and Engineering, Shanghai Jiao Tong University, Shanghai, China. From 2018 to 2019, he was a Research Scholar with the School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore. His research focuses on applied cryptography, big data security, and cloud computing security.



Fagen Li (Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2007.

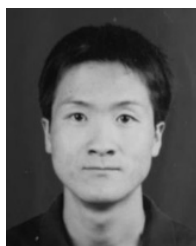
He is currently a Professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu, China. From 2008 to 2009, he was a Postdoctoral Research Fellow with Future University Hakodate, Hakodate, Japan. From 2010 to 2012, he served as a Research Fellow with the Institute of Mathematics for Industry, Kyushu

University, Fukuoka, Japan. His current research focuses on cryptographic protocols and network security systems.



Ximin Jing received the B.Sc. degree from Chengdu University of Information Technology, Chengdu, China, in 2023. He is currently pursuing the M.Sc. degree in cyberspace security with the School of Computer and Software Engineering, Southwest Petroleum University, Chengdu.

His research focuses on redactable blockchain applications and big data auditing.



Yu Zhou received the B.Sc. and M.S. degrees in mathematics from Lanzhou University, Lanzhou, China, in 2003 and 2006, respectively, and the Ph.D. degree in communication engineering from Xidian University, Xi'an, China, in 2009.

Since 2010, he has been with the Science and Technology on Communication Security Laboratory, Chengdu, China. His research focuses on sequence design, coding theory, and cryptographic protocols.